Playmaker: computer offence

Code at commit c7da7b9 2026-11-05. Built 2026-11-05. Line numbers are the real ones in each file.

Everything the computer uses to choose the **offence's** actions. The computer sees only what a human player sees (the Coach's Call target is masked for the defence, and card ratings are read at the level's card reading).

The path of a decision. `Computer.choose` → `_attack`. On a play turn, `_advantage_attack` generates advantage plays (`_adv_plays`), scores shots by the shot bars (`_bar`, `_contested`) and every other play by create, reply, exploit (`_adv_value`, `_exploit`, `pass_risk`), and returns the best. If nothing scores above zero, `_attack` falls back to the older valuation of the generator's plays (`_play_value`, step 3f). `_plan_note` keeps the plan (the advantage play being worked) and writes the log notes the review reads.

Contents

Section	Lines	What it is
computer.py (shared core)	517	The Computer framework, card reading, reachable contests.
solver.py (shared core)	730	The evaluation both sides use, defender reach, applying plays.
solver_config.py	49	Evaluation widths and weights.
computer_levels.json (shared keys)	48	Thinking time, level names.
computer.py (offence)	693	Targets, shot bars, passing, advantage plays, the plan.
advantage.py	228	Beaten, mismatch, help pulled, bad closeout, spent, numbers.
solver.py (offence plays)	67	Offence candidate plays for the fallback search.
computer_levels.json (offence)	185	The offence presets: every lever.
policies.py	156	The base class; the greedy baseline.
solver.py (search, appendix)	260	The search; not used by any level now.

Part 1. The shared core (the same in both computer documents)

The computer player framework, card reading, contest reach, and the solver's evaluation and reach model that both sides use to value positions.

playmaker_engine/computer.py

Shared: the Computer class framework (choose, queue, values, ranking), card reading by level, reachable contests.

```python
"""The computer opponent, both sides, at three levels (step 3e).

The computer chooses among legal actions only; no rule changes. It sees what a human player sees: the
cards (read exactly, by their marks, or by position, by level), the board and the trackers. It never
reads the Coach's Call target when defending, gets no extra tokens, and uses no stored solver results.

How it values things:
- A contest is valued after the defence's legal reaction (`reachable_contest`): a defender with their
  Move unused, not locked and a token in the pool can still get adjacent and facing once the shot is
  declared; Quick Step keeps the facing, a Sprint faces the way it ran.
- A position is valued by the offence's best next action against it (shoot now, drive, or pass and
  then shoot), the solver's evaluation. A level may instead use the solver's search at a short budget
  (lookahead 'solver'); calibration found the one-turn ranking stronger within the time limit.
- Concepts, habits, tunnel vision, card reading, the comfort line and randomness are the level levers
  (computer_levels.json).
"""
import gc
import json
import os
import random
import time

from . import board as B
from . import rules_config as RC
from .actions import Action, legal_actions, MOVEMENT
from .cards import CARDS
from .policies import Policy, expected
from .resolve import apply, ChanceNode
from .state import Context

with open(os.path.join(os.path.dirname(__file__), 'computer_levels.json')) as _f:
    LEVELS = json.load(_f)
LEVEL_NAMES = ('rookie', 'starter', 'allstar')
BUDGET = dict(LEVELS['budget_s'])          # 'server' / 'browser'; the browser build lowers 'active'
BUDGET['active'] = BUDGET['server']


# ---- the reaction (section 1) ----------------------------------------------------------------

def reaction_budget(state):
    """Actions the defence may spend reacting to a declared shot: 0, 1 or 2 (OQ-6), from its tokens."""
    return min(2, state.tok_x) if state.tok_x > 0 else 0


def reachable_contest(state, shooter=None, shot=None, budget=None):
    """Every defender who could contest a shot by `shooter` (default: the ball carrier) from where the
    shooter stands, after the defence's legal reaction. Each entry: defender, clean (arrow on the shooter
    already, or the reaction gets them adjacent and facing), actions (reaction actions needed), contest
    (rating after the -3 when further from the rim than the shooter), value (shot points after this
```

```
    contest; a foul roll entry also gives the foul probability). The reaction budget is shared: the
    contest is made by one defender, who may use the whole budget."""
    from .solver import defender_reach
    from .scoring import foul_on_shot_points
    shooter = state.ball if shooter is None else shooter
    spos = state.o[shooter]
    z = B.zone(spos)
    types = RC.ZONE_SHOTS[z]
    if not types:
        return []
    t = shot or max(types, key=lambda k: state.ctx.off[shooter].shot(k))
    pts = RC.ZONE_POINTS[z]
    card = state.ctx.off[shooter]
    budget = reaction_budget(state) if budget is None else budget
    rules = state.ctx.rules
    if rules.contest_geometry != 'current' or rules.arrow_mode != 'single':
        return _reachable_contest_geo(state, shooter, spos, t, pts, card, budget)
    out = []
    for d in range(5):
        free, fixed = defender_reach(state, d, budget)
        best = None
        for q in set(free) | set(fixed):
            if not B.adjacent(q, spos) or q in state.o:
                continue
            on = B.dir_between(q, spos)
            far = B.dist_rim(q) > B.dist_rim(spos)
            c = state.ctx.dfn[d].con(t) - (RC.FAR_CONTEST_PENALTY if far else 0)
            if state.ctx.rules.floor_contest_zero:
                c = max(0, c)
            clean_len = len(free[q]) if q in free else None
            for a, seq in fixed.get(q, {}).items():
                if a == on and (clean_len is None or len(seq) < clean_len):
                    clean_len = len(seq)
            if clean_len is not None:
                cand = dict(defender=d, square=B.sq(q), clean=True, actions=clean_len, contest=c,
                            value=max(0, pts * (card.shot(t) - c)))
            else:
                ln = min(len(seq) for seq in fixed[q].values())
                pf = state.ctx.dfn[d].r['foul_contest'] / 20.0
                fp = float(foul_on_shot_points(card, t, pts, state.ctx.rules)[0])
                v = max(0, pts * (card.shot(t) - c))
                cand = dict(defender=d, square=B.sq(q), clean=False, actions=ln, contest=c, foul_p=pf,
                            value=pf * fp + (1 - pf) * v)
            if best is None or (cand['value'], cand['actions']) < (best['value'], best['actions']):
                best = cand
        if best:
            out.append(best)
    return sorted(out, key=lambda e: e['value'])


def _reachable_contest_geo(state, shooter, spos, t, pts, card, budget):
    """reachable_contest under a step 3h setting (a wedge, or a positional contest geometry), through
    geometry.best_terms: each entry also names its category (front / side / behind)."""
    from . import geometry as G
    from .solver import defender_reach
    from .scoring import foul_on_shot_points
    rules = state.ctx.rules
    out = []
    for d in range(5):
        free, fixed = defender_reach(state, d, budget)
        best = None
        for q in set(free) | set(fixed):
            if not B.adjacent(q, spos) or q in state.o:
                continue
            opts = []
            if q in free:
                opts.append((None, len(free[q])))
            opts += [((a,), len(seq)) for a, seq in fixed.get(q, {}).items()]
            for arrows, ln in opts:
                bt = G.best_terms(rules, q, arrows, spos)
                if bt is None:
                    continue
                _, foul_roll, penalty, cat, _ = bt
                c = state.ctx.dfn[d].con(t) - (RC.FAR_CONTEST_PENALTY if penalty else 0)
                if rules.floor_contest_zero:
                    c = max(0, c)
                v = max(0, pts * (card.shot(t) - c))
                if foul_roll:
                    pf = state.ctx.dfn[d].r['foul_contest'] / 20.0
                    fp = float(foul_on_shot_points(card, t, pts, rules)[0])
                    cand = dict(defender=d, square=B.sq(q), clean=False, actions=ln, contest=c, foul_p=pf,
                                value=pf * fp + (1 - pf) * v, category=cat)
                else:
                    cand = dict(defender=d, square=B.sq(q), clean=True, actions=ln, contest=c, value=v, category=cat)
                if best is None or (cand['value'], cand['actions']) < (best['value'], best['actions']):
                    best = cand
        if best:
            out.append(best)
    return sorted(out, key=lambda e: e['value'])


# ---- passing judgement (step 3f) ----------------------------------------------------------------

_PASS_CACHE = {}
```

… lines 146–217: printed in the offence document …

```

```

… lines 220–233: printed in the offence document …

```python
# ---- what a side can see; card reading (section 8 lever 4) ----------------------------------------

def _positions():
    from .scouting import position
    return {k: position(c) for k, c in CARDS.items()}


_POS = None
_MIDS = None


def _mids():
    """Middle value of each band of each rating, and each position's typical band and QS, from the 75 cards."""
    global _MIDS, _POS
    if _MIDS is None:
        _POS = _positions()
        vals, by_pos = {}, {}
        for c in CARDS.values():
            for r, v in c.r.items():
                vals.setdefault((r, c.bands[r]), []).append(v)
                by_pos.setdefault((_POS[c.key], r), []).append(c.bands[r])
        mid = {k: round(sum(v) / len(v)) for k, v in vals.items()}
        typical = {k: max(set(v), key=v.count) for k, v in by_pos.items()}
        qs = {}
        for c in CARDS.values():
            qs.setdefault(_POS[c.key], []).append(c.qs)
        qs = {p: sorted(v)[len(v) // 2] for p, v in qs.items()}
        _MIDS = (mid, typical, qs)
    return _MIDS


class SeenCard:
    """A card as the computer reads it at its level: 'exact' (the numbers), 'marks' (only ▲ / plain / ▼:
    every rating is the middle of its band) or 'role' (judged by position: the position's typical band)."""
    __slots__ = ('key', 'num', 'role', 'tier', 'tagline', 'qs', 'r', 'bands')

    def __init__(self, card, reading):
        mid, typical, qs = _mids()
        self.key, self.num, self.role, self.tier, self.tagline = card.key, card.num, card.role, card.tier, card.tagline
        if reading == 'marks':
            self.bands = dict(card.bands)
            self.qs = card.qs
        else:                                   # role
            pos = _POS[card.key]
            self.bands = {r: typical[(pos, r)] for r in card.r}
            self.qs = qs[pos]
        self.r = {r: mid[(r, b)] for r, b in self.bands.items()}

    def shot(self, t):
        return self.r['shot_' + t]

    def con(self, t):
        return self.r['con_' + t]


def seen_context(ctx, reading):
    if reading == 'exact':
        return ctx
    return Context([SeenCard(c, reading) for c in ctx.off], [SeenCard(c, reading) for c in ctx.dfn], ctx.rules,
                   ctx.title, ctx.kind, ctx.scenario, ctx.card_clock, ctx.inbounder, ctx.coach_call,
                   ctx.three_only, ctx.two_or_three, ctx.hold_for_one, ctx.off_team, ctx.def_team)


VISIBLE_FIELDS = ('o', 'x', 'arrows', 'ball', 'clock', 'phase', 'side', 'stage', 'tok_o', 'tok_x', 'pending',
                  'shot', 'screens', 'locks', 'moved', 'counts')


def seen_state(state, side, ctx):
    """The state as `side` may see it: the defence never sees the offence's secret Coach's Call target."""
    s = state.with_(ctx=ctx)
    if side == 'X' and state.ctx.coach_call and state.stage != 'over':
        s = s.with_(target=-1)
    return s


# ---- the level evaluation: the solver's evaluation, with the concepts as switches -----------------

def _evaluator_class():
    from .solver import Evaluator

    class LevelEvaluator(Evaluator):
        """reaction: count reaction-reachable contests (off: only the arrows as they stand);
        reserve: see that a spent token or Move loses the reaction (off: always assume a reaction);
        tunnel: only the N attackers nearest the ball are a threat."""

        def __init__(self, cfg, reaction=True, reserve=True, tunnel=5):
            super().__init__(cfg)
            self.reaction, self.reserve, self.tunnel = reaction, reserve, tunnel

        def reach(self, s, budget):
            if not self.reaction:
                budget = 0
            elif not self.reserve:
                s = s.with_(moved_x=0, tok_x=max(s.tok_x, 2))
                budget = 2
            return super().reach(s, budget)

```

```
        def visible(self, s):
            if self.tunnel >= 5:
                return None
            cp = s.o[s.ball]
            order = sorted(range(5), key=lambda i: (i != s.ball, B.cheb(s.o[i], cp), i))
            return set(order[:self.tunnel])

        def carrier_look(self, s, shooter, budget_x, can_move, forced=False):
            vis = self.visible(s)
            if vis is not None and shooter not in vis:
                return (0.0, 0.0, 0.0, None, None)
            return super().carrier_look(s, shooter, budget_x, can_move, forced)

    return LevelEvaluator


# ---- the computer player ---------------------------------------------------------------------

class Computer(Policy):
    """One side's computer at a level. choose(state) -> a legal action. notes: decisions to log (plans,
    targets, habits that fired)."""
    name = 'computer'

    def __init__(self, side, level='starter', seed=0, overrides=None, habits=None, budget_s='active'):
        if level not in LEVEL_NAMES:
            raise ValueError(f'level must be one of {LEVEL_NAMES}')
        self.side, self.level = side, level
        part = LEVELS['defence' if side == 'X' else 'offence']
        self.cfg = json.loads(json.dumps(part['levels'][level]))
        for k, v in (overrides or {}).items():
            if isinstance(v, dict) and isinstance(self.cfg.get(k), dict):
                self.cfg[k].update(v)
            else:
                self.cfg[k] = v
        self.rng = random.Random(seed * 7919 + (1 if side == 'O' else 2))
        pool = [h for h in part['habits'] if h not in self.cfg.get('habits_not', [])]
        self.habits = list(habits) if habits is not None else self.rng.sample(pool, self.cfg['habits'])
        self.budget_s = BUDGET['active'] if budget_s == 'active' else budget_s
        self.comfort = (LEVELS['defence']['comfort_line'] + self.cfg.get('comfort_offset', 0)) if side == 'X' else None
        self.notes = []
        self.stats = dict(decisions=0, comfort_decided=0, habit_fired={}, seconds=[])
        self._ctx = None
        self.plan = None
        self.queue = None

    # -- setup per possession (the solver's caches key positions without their context) --
    def _context(self, state):
        if state.ctx is self._ctx:
            return
        from .solver import Generator, Searcher
        from .solver_config import with_config
        self._ctx = state.ctx
        self.seen = seen_context(state.ctx, self.cfg['reading'])
        c = self.cfg['concepts']
        reaction = c.get('reaction', c.get('reaction_shot', True))
        reserve = c.get('reserve', True) if self.side == 'X' else c.get('drain', True)
        # no root-wide raw action scan and no exact endgame hand-off: neither watches the clock
        self.ecfg = with_config(eval_turns=1, top_k_raw=0, endgame_tokens=-1, eval_reply_k=2)
        self.ev = _evaluator_class()(self.ecfg, reaction=reaction, reserve=reserve, tunnel=self.cfg['tunnel'])
        self.gen = Generator(self.ecfg, self.ev)
        self.ev._gen = self.gen
        self.Searcher = Searcher
        self.queue = None
        self.plan = None
        self.visited = {}           # offence: (squares, ball) reached at its decisions this possession -> times
        self.aplan = None           # offence: the advantage play being worked (step 3g)
        self._handover = None       # offence: the board as it left it to the defence (for help_pulled)
        self.picked_off = set()     # offence: passes intercepted this possession, keyed by the full position
        self._last_pass = None

    def describe(self):
        """For the log and the review: level, habits drawn, concepts switched off."""
        off = [k for k, v in self.cfg['concepts'].items() if not v and k != 'reaction_shot']
        return dict(level=self.level, name=LEVELS['names'][self.level][0], habits=self.habits, concepts_off=off)

    def _fire(self, habit):
        self.stats['habit_fired'][habit] = self.stats['habit_fired'].get(habit, 0) + 1

    # -- choose --
    def choose(self, state):
        """A legal action. Python's cycle collector is paused while it thinks: the search's caches hold
        millions of small positions, and a full collection walking them cost up to 5 seconds a decision.
        Positions form no reference cycles, so reference counting still frees them as it goes."""
        was = gc.isenabled()
        gc.disable()
        try:
            return self._choose(state)
        finally:
            if was:
                gc.enable()

    def _choose(self, state):
        acts = legal_actions(state)
        if len(acts) == 1:
            return acts[0]
        self._context(state)
        if self.side == 'O' and state.stage == 'play' and state.side == 'O':
```

```
            self._remember(state)
        t0 = time.time()
        self._deadline = t0 + self.budget_s if self.budget_s else None
        if self.queue and self.queue[0] == state and self.queue[1] and self.queue[1][0] in acts:
            a = self.queue[1][0]
            self._queue(state, a, self.queue[1][1:])
            self._note_pass(state, a)
            self._note_handover(state, a)
            return a
        self.queue = None
        view = seen_state(state, self.side, self.seen)
        if self.side == 'X':
            play = self._defend(view, acts)
        else:
            play = self._attack(view, acts)
        a = play.actions[0] if play is not None else acts[0]
        if a not in acts:
            a = acts[0]
        self._queue(state, a, play.actions[1:] if play is not None else ())
        self._note_pass(state, a)
        self._note_handover(state, a)
        self.stats['decisions'] += 1
        self.stats['seconds'].append(time.time() - t0)
        return a

    # -- offence memory over the possession: no loops, no repeating a pass that was just picked off --
… lines 456–458: printed in the offence document …

… lines 460–465: printed in the offence document …

… lines 467–469: printed in the offence document …

… lines 471–479: printed in the offence document …
    def _queue(self, state, a, rest):
        nxt = apply(state, a)
        self.queue = (nxt, tuple(rest)) if rest and not isinstance(nxt, ChanceNode) else None

    # -- values --
    def _immediate(self, s):
        """Lookahead 0: the ball carrier's shot as it stands (after the reaction, per the concepts)."""
        if s.stage != 'play' or s.pending:
            return self.ev(s)[0].ev
        base = RC.STEAL_FOUL_BONUS * s.bonus5 + RC.TURNOVER_POINTS * s.tos
        ev = self.ev.carrier_look(s, s.ball, reaction_budget(s), False)[0]
        return base + ev

    def _node_value(self, node, fn):
        if isinstance(node, ChanceNode):
            if self.side == 'O' and not self.cfg['concepts'].get('risk', True):
                return max(self._node_value(sub, fn) for _, _, sub in node.outcomes)
            return sum(float(p) * self._node_value(sub, fn) for p, _, sub in node.outcomes)
        return fn(node)

    def _value_fn(self):
        la = self.cfg['lookahead']
        if la == 0:
            return self._immediate
        if self.side == 'X':
            return self._threat
        return lambda s: (self.ev._reply(s)[0].ev if s.stage == 'play' and s.side == 'X' else self.ev(s)[0].ev)
… lines 509–520: printed in the defence document …
    def _rank(self, view, plays, fn, maximise, stop):
        from .solver import apply_play
        scored = []
        for p in plays:
            if stop is not None and scored and time.time() > stop:
                break
            node = apply_play(view, p)
            if node is None:
                continue
            v = self._node_value(node, fn)
            scored.append((v, p))
        return scored

    def _pick(self, scored, maximise, key=None):
        """Best, or with the level's randomness the second best (never the main source of mistakes)."""
        if not scored:
            return None
        cost = lambda p: sum(a.kind != 'end_turn' for a in p.actions) * 1e-6
        k = key or (lambda vp: (-vp[0] if maximise else vp[0]) + cost(vp[1]))
        ranked = sorted(scored, key=k)
        if len(ranked) > 1 and self.rng.random() < self.cfg['randomness']:
            return ranked[1][1]
        return ranked[0][1]

    @staticmethod
    def _with_end(view, acts, plays):
        """Mid-turn (one action already taken), stopping is a choice too: the generator builds plays for a
        fresh turn and leaves it out."""
        from .solver import Play
        end = next((a for a in acts if a.kind == 'end_turn'), None)
        if end is not None and view.stage == 'play' and view.turn_n >= 1 and not any(p.actions == (end,) for p in plays):
            plays = list(plays) + [Play('end', (end,))]
        return plays

    def _stop(self):
```

```
        return getattr(self, '_deadline', None)

    def _late(self, share=0.85):
        """Most of the thinking time already gone (the search used it): fall back to the quick value."""
        d = self._stop()
        return d is not None and self.budget_s and time.time() > d - self.budget_s * (1 - share)

    # ---- defence ----
… lines 565–604: printed in the defence document …

… lines 606–620: printed in the defence document …

… lines 622–648: printed in the defence document …

… lines 650–671: printed in the defence document …

… lines 673–683: printed in the defence document …

… lines 685–693: printed in the defence document …
    def _search(self, view):
        """The solver's search, deepening until the thinking time is up (a hard stop: when not even depth
        1 finished, the caller falls back to the one-turn ranking with what time is left)."""
        from .solver import Timeout
        se = self.Searcher(self.ecfg)
        se.ev = self.ev
        se.gen = self.gen
        budget = self.budget_s or 0.5
        se.deadline = time.time() + budget * 0.5
        best = None
        try:
            for d in range(1, self.ecfg.max_depth + 1):
                se.root_depth = d
                se.value(view, d)
                e = se.tt.get((view, d))
                if e is not None and e.best is not None:
                    best = e.best
        except Timeout:
            pass
        return best

    # ---- offence ----
… lines 717–745: printed in the offence document …

… lines 747–768: printed in the offence document …

… lines 770–782: printed in the offence document …

… lines 784–859: printed in the offence document …
    # -- passing judgement (step 3f) --
… lines 862–863: printed in the offence document …

… lines 865–889: printed in the offence document …

… lines 891–903: printed in the offence document …

… lines 905–907: printed in the offence document …

… lines 909–915: printed in the offence document …

… lines 917–933: printed in the offence document …

… lines 935–956: printed in the offence document …

… lines 958–992: printed in the offence document …
    # ---- step 3g: create an advantage, keep it, finish it -------------------------------------------
… lines 995–995: printed in the offence document …

… lines 997–998: printed in the offence document …

… lines 1000–1002: printed in the offence document …

… lines 1004–1012: printed in the offence document …

… lines 1014–1018: printed in the offence document …

… lines 1020–1045: printed in the offence document …

… lines 1047–1051: printed in the offence document …

… lines 1053–1085: printed in the offence document …

… lines 1087–1154: printed in the offence document …

… lines 1156–1222: printed in the offence document …
    # -- the plan: the advantage play being worked, kept until finished or taken away --
… lines 1225–1246: printed in the offence document …

… lines 1248–1288: printed in the offence document …

… lines 1290–1297: printed in the offence document …
```

```
1298
    … lines 1299–1301: printed in the offence document …
1302
    … lines 1303–1304: printed in the offence document …
1305
    … lines 1306–1314: printed in the offence document …
1315
    … lines 1316–1321: printed in the offence document …
1322
    … lines 1323–1334: printed in the offence document …
1335 def make(side, level, seed=0, **kw):
1336     return Computer(side, level, seed, **kw)
1337
1338
1339 # ---- setup and placement (section 5) ------------------------------------------------------------
    … lines 1343–1347: printed in the defence document …
1348
    … lines 1350–1431: printed in the defence document …
1432
```

playmaker_engine/solver.py

Shared: position values (Evaluator), defender reach, plays and how they apply.

```
   1 """Step 2: the scenario solver.
   2
   3 Three parts, all on top of the unchanged engine:
   4   * exact_solve   : expectimax over raw engine actions with a transposition table (exact).
   5   * evaluate      : fast position value: the best look the offence can get on its next turn
   6                     against the defence's best two-action reaction (the regression ceiling, generalised).
   7   * Searcher      : depth-limited expectimax over basketball plays (candidate generator) with the
   8                     evaluation at the leaves, iterative deepening under a time budget, exact
   9                     probabilities at chance nodes, and the exact solver for the tail.
  10 Values are per Coach's Call target (a tuple); ordinary cards have one entry.
  11 """
  12 import random
  13 import time
  14 from dataclasses import dataclass, field
  15
  16 from . import board as B
  17 from . import geometry as G
  18 from . import rules_config as RC
  19 from .actions import Action, legal_actions, movement_dests, can_move_kind, MOVEMENT, intercept_status, t_square
  20 from .resolve import apply, ChanceNode
  21 from .scoring import score, contest_rating, foul_on_shot_points, p_success, band
  22 from .solver_config import DEFAULT
  23 from .state import CI
  24
  25
  26 class Timeout(Exception):
  27     pass
  28
  29
  30 # ============================================================================
  31 # values
  32 # ============================================================================
  33
  34 @dataclass(frozen=True)
  35 class Val:
  36     """Expected possession score, expected look, P(look >= 30), P(clock reaches 0),
  37     expected forced-shot points (counted only when the clock expires)."""
  38     ev: float
  39     look: float = 0.0
  40     p30: float = 0.0
  41     pexp: float = 0.0
  42     fs: float = 0.0
  43
  44
  45 def mix(pairs):
  46     """Weighted average of Val tuples: pairs = [(p, (Val, ...)), ...]."""
  47     n = len(pairs[0][1])
  48     out = []
  49     for t in range(n):
  50         ev = look = p30 = pexp = fs = 0.0
  51         for p, vs in pairs:
  52             v = vs[t]
  53             ev += p * v.ev
  54             look += p * v.look
  55             p30 += p * v.p30
  56             pexp += p * v.pexp
  57             fs += p * v.fs
  58         out.append(Val(ev, look, p30, pexp, fs))
  59     return tuple(out)
  60
  61
  62 def terminal_vals(s, targets):
  63     out = []
  64     for t in targets:
  65         st = s if t is None else s.with_(target=t)
  66         sc = score(st)
  67         r = s.result
  68         shot = r is not None and r.kind in ('shot', 'forced', 'free_throws')
  69         look = float(sc.look) if shot else 0.0
  70         expired = s.clock == 0
  71         out.append(Val(float(sc.total), look, 1.0 if (shot and sc.look >= RC.SCENARIO_GOAL and not s.zeroed) else 0.0,
```

```
                      1.0 if expired else 0.0, look if expired else 0.0))
    return tuple(out)


# ============================================================================
# fast evaluation
# ============================================================================

def _with_x(s, d, q, arrow=None):
    x = list(s.x)
    x[d] = q
    kw = dict(x=tuple(x))
    if arrow is not None:
        arr = list(s.arrows)
        arr[d] = arrow
        kw['arrows'] = tuple(arr)
    return s.with_(**kw)


from functools import lru_cache


@lru_cache(maxsize=None)
def _paths(kind, frm, qs, rules):
    """On-board movement paths from frm: tuple of (run direction or None, path, path as frozenset, dest)."""
    if kind == 'move':
        raw = [(None, p) for p in B.move_paths(frm, rules)]
    elif kind == 'qs':
        raw = [(None, p) for p in B.qs_paths(frm, qs, rules)]
    else:
        raw = list(B.sprint_paths(frm, qs, rules))
    return tuple((dr, p, frozenset(p), p[-1]) for dr, p in raw if all(B.on_board(q) for q in p))


def defender_reach(s, d, budget):
    """Where defender d can be after up to `budget` (<= 2) actions, and with what arrow.
    Returns (free, fixed): free[sq] = action list arriving with any arrow (a Move was used);
    fixed[sq] = {arrow: action list} for arrivals whose arrow is set (Quick Step / Sprint).
    Uses the engine's movement shapes, blocking and screen geometry directly (no state copies)."""
    rules = s.ctx.rules
    start = s.x[d]
    arrow = s.arrows[d]
    free, fixed = {}, {start: {arrow: []}}
    if budget <= 0 or s.tok_x <= 0:
        return free, fixed
    qs = s.ctx.dfn[d].qs
    occ = (set(s.o) | set(s.x)) - {start}
    scr = []
    for sc in s.screens:
        scr.append((t_square(s, sc), bool(sc[3] >> d & 1)))
    qs_left = RC.LIMITS['X']['qs'] - s.count('X', 'qs')
    once = rules.qs_once_per_player
    if once and s.quick_stepped('X', d):
        qs_left = 0                          # OQ-52: this defender's Quick Step is used (no Quick Step, no Sprint)
    if once:
        qs_left = min(qs_left, 1)            # one Quick Step per player: never two, nor a Quick Step and a Sprint
    can_move = not s.moved('X', d) and not s.locked('X', d)
    two = budget >= 2 and s.tok_x >= 2
    on_board, banned = B.on_board, B.diag_step_banned

    def ok(frm, path):
        if scr:
            prev = frm
            for q in path:
                for tsq, beat in scr:
                    if not beat and banned(prev, q, tsq, rules):
                        return False
                prev = q
            for tsq, beat in scr:
                if beat and path[-1] == tsq:
                    return False
        return True

    def dests(frm, kind):
        out = {}
        disjoint = occ.isdisjoint
        for dr, path, pset, dest in _paths(kind, frm, qs, rules):
            if dest not in out and disjoint(pset) and (not scr or ok(frm, path)):
                out[dest] = dr
        return out

    def add_fixed(q, a, seq):
        cur = fixed.setdefault(q, {})
        if a not in cur or len(seq) < len(cur[a]):
            cur[a] = seq

    def add_free(q, seq):
        if q not in free or len(seq) < len(free[q]):
            free[q] = seq

    moves = dests(start, 'move') if can_move else {}
    qss = dests(start, 'qs') if qs_left >= 1 and qs > 0 else {}
    sprints = dests(start, 'sprint') if can_move and qs_left >= 1 else {}
    if can_move:
        add_free(start, [('move', start)])
    for q in moves:
        add_free(q, [('move', q)])
```

```
    for q in qss:
        add_fixed(q, arrow, [('qs', q)])
    wedge = rules.arrow_mode == 'wedge'
    for q, dr in sprints.items():
        add_fixed(q, dr, [('sprint', q)])
        if wedge:                           # step 3h: the other edge on the anticlockwise side
            add_fixed(q, (dr - 1) % 8, [('sprint', q, (dr - 1) % 8)])
    if two:
        if qs_left >= 1 and qs > 0:
            for m in list(moves) + ([start] if can_move else []):
                for q in dests(m, 'qs'):
                    add_free(q, [('move', m), ('qs', q)])
        for q1 in qss:
            if can_move:
                for q in dests(q1, 'move'):
                    add_free(q, [('qs', q1), ('move', q)])
            if qs_left >= 2:
                for q in dests(q1, 'qs'):
                    add_fixed(q, arrow, [('qs', q1), ('qs', q)])
                if can_move:
                    for q, dr in dests(q1, 'sprint').items():
                        add_fixed(q, dr, [('qs', q1), ('sprint', q)])
                        if wedge:
                            add_fixed(q, (dr - 1) % 8, [('qs', q1), ('sprint', q, (dr - 1) % 8)])
        if qs_left >= 2 and qs > 0:
            for q1, dr1 in sprints.items():
                for q in dests(q1, 'qs'):
                    add_fixed(q, dr1, [('sprint', q1), ('qs', q)])
                    if wedge:
                        add_fixed(q, (dr1 - 1) % 8, [('sprint', q1, (dr1 - 1) % 8), ('qs', q)])
    return free, fixed


_DR = {(x, y): B.dist_rim((x, y)) for x in range(-1, B.W + 1) for y in range(0, B.H + 2)}


class ReachMaps(list):
    """Per-defender (free, fixed) maps, plus an index: square -> [(d, free_len, [(arrow, len)])]."""

    def __init__(self, maps):
        super().__init__(maps)
        idx = {}
        for d, (free, fixed) in enumerate(maps):
            squares = set(free) | set(fixed)
            for q in squares:
                fl = len(free[q]) if q in free else None
                fx = [(a, len(seq)) for a, seq in fixed.get(q, {}).items()]
                idx.setdefault(q, []).append((d, fl, fx))
        self.index = idx


class Evaluator:
    """Fast position values, cached per position."""

    def __init__(self, cfg=DEFAULT, targets=(None,)):
        self.cfg = cfg
        self.targets = targets
        self.reach_cache = {}
        self.cache = {}
        self.calls = 0

    # -- reach of every defender, cached on what it depends on
    def reach(self, s, budget):
        key = (s.o, s.x, s.arrows, s.moved_x, s.cnt[6], s.tok_x, s.lock_x, s.screens, budget)
        r = self.reach_cache.get(key)
        if r is None:
            r = ReachMaps([defender_reach(s, d, budget) for d in range(5)])
            if len(self.reach_cache) > 15000:
                self.reach_cache.clear()
            self.reach_cache[key] = r
        return r

    def defended(self, s, maps, shooter, spos, t, budget, forced=False):
        """Defence's best answer to a shot by `shooter` from spos (shot type t).
        Returns (ev, look, p30, kind) where kind names the answer ('open', 'clean', 'side', 'steal')."""
        if s.ctx.rules.contest_geometry != 'current':
            return self._defended_geo(s, maps, shooter, spos, t, budget, forced)
        ctx, rules = s.ctx, s.ctx.rules
        card = ctx.off[shooter]
        pts = RC.ZONE_POINTS[B.zone(spos)]
        rating = card.shot(t)
        if ctx.three_only and pts != 3:
            return 0.0, 0.0, 0.0, None
        floor = forced or rules.floor_declared_shot
        fp = float(foul_on_shot_points(card, t, pts, rules)[0])
        fp30 = fp >= 30
        v0 = pts * rating
        best = (float(v0), float(v0), 1.0 if v0 >= 30 else 0.0, 'open')
        best_ev = best[0]
        steal_ok = s.count('X', 'steal') < RC.LIMITS['X']['steal']
        tok = s.tok_x
        far_pen = RC.FAR_CONTEST_PENALTY
        floor_c = rules.floor_contest_zero
        ds = _DR[spos]
        idx = maps.index
        occ_o = s.o
        sx, sy = spos
```

```
        pk_cache = {}
        con_key = 'con_' + t
        for dx, dy in B.DIRS:
            q = (sx + dx, sy + dy)
            ents = idx.get(q)
            if not ents or q in occ_o:
                continue
            on = B.DIRS.index((-dx, -dy))
            far = _DR[q] > ds
            for d, fl, fx in ents:
                dcard = ctx.dfn[d]
                c = dcard.r[con_key] - (far_pen if far else 0)
                if floor_c and c < 0:
                    c = 0
                v = pts * (rating - c)
                if floor and v < 0:
                    v = 0
                v = float(v)
                clean_len = None
                if fl is not None and fl <= budget:
                    clean_len = fl
                for a, ln in fx:
                    if ln <= budget and a == on and (clean_len is None or ln < clean_len):
                        clean_len = ln
                if clean_len is not None:
                    if v < best_ev:
                        best, best_ev = (v, v, 1.0 if v >= 30 else 0.0, 'clean'), v
                else:
                    pf = dcard.r['foul_contest'] / 20.0
                    for a, ln in fx:
                        if ln <= budget and a != on:
                            ev = pf * fp + (1 - pf) * v
                            if ev < best_ev:
                                best, best_ev = (ev, ev, pf * fp30 + (1 - pf) * (v >= 30), 'side'), ev
                            break
                if steal_ok and clean_len is not None and clean_len + 1 <= budget and tok >= clean_len + 1:
                    pf = dcard.r['foul_steal'] / 20.0
                    pk = pk_cache.get(d)
                    if pk is None:
                        pk = pk_cache[d] = float(p_success(card.r['dribble'] - dcard.r['steal'], rules))
                    ev = pf * fp + (1 - pf) * (pk * v + (1 - pk) * RC.TURNOVER_POINTS)
                    if ev < best_ev:
                        best = (ev, pf * fp + (1 - pf) * pk * v, pf * fp30 + (1 - pf) * pk * (v >= 30), 'steal')
                        best_ev = ev
        return best

    def _defended_geo(self, s, maps, shooter, spos, t, budget, forced=False):
        """defended() under the step 3h contest geometries: only a defender who ends with the shooter on
        an arrow line contests; Front is a full contest, Side a foul roll, Behind a foul roll and the -3
        (geometry.terms decides, from the arrows each reach path can end with)."""
        ctx, rules = s.ctx, s.ctx.rules
        card = ctx.off[shooter]
        pts = RC.ZONE_POINTS[B.zone(spos)]
        rating = card.shot(t)
        if ctx.three_only and pts != 3:
            return 0.0, 0.0, 0.0, None
        floor = forced or rules.floor_declared_shot
        fp = float(foul_on_shot_points(card, t, pts, rules)[0])
        fp30 = fp >= 30
        v0 = pts * rating
        best = (float(v0), float(v0), 1.0 if v0 >= 30 else 0.0, 'open')
        best_ev = best[0]
        steal_ok = s.count('X', 'steal') < RC.LIMITS['X']['steal']
        tok = s.tok_x
        idx = maps.index
        occ_o = s.o
        sx, sy = spos
        con_key = 'con_' + t
        pk_cache = {}
        for dx, dy in B.DIRS:
            q = (sx + dx, sy + dy)
            ents = idx.get(q)
            if not ents or q in occ_o:
                continue
            for d, fl, fx in ents:
                dcard = ctx.dfn[d]
                opts = []
                if fl is not None and fl <= budget:
                    opts.append((None, fl))
                opts += [((a,), ln) for a, ln in fx if ln <= budget]
                facing_len = None
                for arrows, ln in opts:
                    bt = G.best_terms(rules, q, arrows, spos)
                    if bt is None:
                        continue                            # lines off the shooter: no contest
                    _, foul_roll, penalty, _, _ = bt
                    facing_len = ln if facing_len is None else min(facing_len, ln)
                    c = dcard.r[con_key] - (RC.FAR_CONTEST_PENALTY if penalty else 0)
                    if rules.floor_contest_zero and c < 0:
                        c = 0
                    v = pts * (rating - c)
                    if floor and v < 0:
                        v = 0
                    v = float(v)
                    if foul_roll:
                        pf = dcard.r['foul_contest'] / 20.0
                        ev = pf * fp + (1 - pf) * v
```

```
                        cand = (ev, ev, pf * fp30 + (1 - pf) * (v >= 30), 'side')
                    else:
                        cand = (v, v, 1.0 if v >= 30 else 0.0, 'clean')
                    if cand[0] < best_ev:
                        best, best_ev = cand, cand[0]
                if steal_ok and facing_len is not None and facing_len + 1 <= budget and tok >= facing_len + 1:
                    c = dcard.r[con_key]
                    v = float(max(0, pts * (rating - c)) if floor else pts * (rating - c))
                    pf = dcard.r['foul_steal'] / 20.0
                    pk = pk_cache.get(d)
                    if pk is None:
                        pk = pk_cache[d] = float(p_success(card.r['dribble'] - dcard.r['steal'], rules))
                    ev = pf * fp + (1 - pf) * (pk * v + (1 - pk) * RC.TURNOVER_POINTS)
                    if ev < best_ev:
                        best = (ev, pf * fp + (1 - pf) * pk * v, pf * fp30 + (1 - pf) * pk * (v >= 30), 'steal')
                        best_ev = ev
        return best

    def carrier_look(self, s, shooter, budget_x, can_move, forced=False):
        """Best (ev, look, p30, shot square, type) for `shooter` with the ball."""
        spos = s.o[shooter]
        maps = self.reach(s, budget_x)
        squares = [spos]
        if can_move:
            st = s.with_(ball=shooter)
            for kind in ('move', 'qs', 'sprint'):
                if can_move_kind(st, 'O', shooter, kind):
                    squares += list(movement_dests(st, 'O', shooter, kind, s.ctx.rules))
        best = (0.0, 0.0, 0.0, None, None)
        first = True
        for q in dict.fromkeys(squares):
            if not B.on_board(q):
                continue
            z = B.zone(q)
            for t in RC.ZONE_SHOTS[z]:
                if q != spos:
                    sh = s.with_(o=s.o[:shooter] + (q,) + s.o[shooter + 1:])
                else:
                    sh = s
                ev, look, p30, _ = self.defended(sh, maps, shooter, q, t, budget_x, forced)
                if first or ev > best[0]:
                    best = (ev, look, p30, q, t)
                    first = False
        return best

    def __call__(self, s):
        key = s
        v = self.cache.get(key)
        if v is not None:
            return v
        self.calls += 1
        v = self._eval(s)
        if self.cfg.eval_turns >= 2 and s.stage == 'play' and s.side == 'O' and s.tok_o >= 1 and not s.ctx.coach_call:
            v2 = self._two_turns(s, v)
            if v2 is not None and v2[0].ev > v[0].ev:
                v = v2
        v = self._terms(s, v)
        if len(self.cache) > 60000:
            self.cache.clear()
        self.cache[key] = v
        return v

    def _eval(self, s):
        if s.stage == 'over':
            return terminal_vals(s, self.targets)
        if s.stage in ('pick', 'forced'):
            return max((expected(apply(s, a), self) for a in legal_actions(s)), key=lambda vs: vs[0].ev)
        if s.stage == 'contest':
            return min((expected(apply(s, a), self) for a in legal_actions(s)), key=lambda vs: max(v.ev for v in vs))
        if s.stage == 'target':
            return max((self(apply(s, a)) for a in legal_actions(s)), key=lambda vs: vs[0].ev)
        ctx = s.ctx
        base = RC.STEAL_FOUL_BONUS * s.bonus5 + RC.TURNOVER_POINTS * s.tos
        if s.pending:
            from .actions import completed_view
            landed = completed_view(s)
            p = 1.0
            for j in range(5):
                if intercept_status(s, j) in ('direct', 'approach'):
                    pj = float(p_success(ctx.off[s.pending[0]].r['pass'] - ctx.dfn[j].r['intercept'], ctx.rules))
                    p = min(p, pj)
            if s.inbound and s.pending[0] == ctx.inbounder:
                from .resolve import complete_pass
                landed = complete_pass(s)
            lost = s.with_(pending=None, approach=-1, tos=s.tos + 1)
            if p >= 1.0:
                return self(landed)
            if ctx.hold_for_one:
                lost_v = tuple(Val(0.0) for _ in self.targets)
            else:
                lost_v = self(lost)
            return mix([(p, self(landed)), (1 - p, lost_v)])
        budget_x = min(2, s.tok_x) if s.tok_x > 0 else 0
        # the shot already declared: the defence's reaction is all that is left
        if s.stage == 'reaction':
            maps = self.reach(s, budget_x)
            ev, look, p30, _ = self.defended(s, maps, s.ball, s.o[s.ball], s.shot, budget_x)
```

```
            return self._vals(s, base, ev, look, p30, s.ball, 0.0, 0.0)
        # clock about to run out: forced shot from where the carrier stands
        last_turn = s.inbound == 0 and s.clock <= RC.PHASE_SECONDS and (
            (s.side == 'X' and s.final) or (s.side == 'O' and s.tok_o == 0))
        if last_turn:
            if ctx.hold_for_one:
                return tuple(Val(0.0, 0.0, 0.0, 1.0, 0.0) for _ in self.targets)
            ev, look, p30, _, _ = self.carrier_look(s, s.ball, budget_x, False, forced=True)
            return self._vals(s, base + RC.TURNOVER_POINTS, ev, look, p30, s.ball, 1.0, ev)
        o_tokens = s.tok_o
        can_move = o_tokens >= 2 or s.side == 'X'
        if s.inbound and s.ball == ctx.inbounder:
            w, cands = self.cfg.inbound_kick_weight, [i for i in range(5) if i != s.ball and B.on_board(s.o[i])]
            # the ball still has to come in: weigh each receiver's look by the pass's interception risk
            best = (RC.TURNOVER_POINTS * 1.0, 0.0, 0.0, -1)
            can_pass = s.count('O', 'pass') < RC.LIMITS['O']['pass'] and s.tok_o > 0
            for r in cands if can_pass else []:
                ev, look, p30, _, _ = self.carrier_look(s, r, budget_x, True)
                pend = s.with_(pending=(s.ball, r))
                p = 1.0
                for j in range(5):
                    if intercept_status(pend, j) in ('direct', 'approach'):
                        p = min(p, float(p_success(ctx.off[s.ball].r['pass'] - ctx.dfn[j].r['intercept'], ctx.rules)))
                v = (p * ev + (1 - p) * RC.TURNOVER_POINTS) * w
                if v > best[0]:
                    best = (v, p * look * w, p * p30 * w, r)
            return self._vals(s, base, best[0], best[1], best[2], best[3], 0.0, 0.0)
        ev, look, p30, _, _ = self.carrier_look(s, s.ball, budget_x, can_move)
        shooter = s.ball
        if self.cfg.kick_weight > 0:
            for r in range(5):
                if r != s.ball and B.on_board(s.o[r]):
                    e2, l2, p2, _, _ = self.carrier_look(s, r, budget_x, True)
                    if e2 * self.cfg.kick_weight > ev:
                        ev, look, p30, shooter = e2 * self.cfg.kick_weight, l2 * self.cfg.kick_weight, p2 * self.cfg.kick_weight, r
        return self._vals(s, base, ev, look, p30, shooter, 0.0, 0.0)

    def _one(self, s):
        """One-turn evaluation (no recursion), cached."""
        k = ('one', s)
        v = self.cache.get(k)
        if v is None:
            v = self._eval(s)
            self.cache[k] = v
        return v

    def _two_turns(self, s, v1):
        """Best look over the offence's next two turns: a setup play now (no shot), the defence's
        best reply among its ball-side plays, then the one-turn evaluation."""
        gen = getattr(self, '_gen', None)
        if gen is None:
            gen = self._gen = Generator(self.cfg, self)
        pri = {'kick': 0, 'relocate+kick': 1, 'drive': 2, 'seal': 3, 'relocate': 4, 'screen': 5, 'swing': 6,
               'dribble': 7, 'hesitate': 8}
        setups = [p for p in gen.offence_plays(s) if not any(a.kind == 'shot' for a in p.actions)]
        setups.sort(key=lambda p: pri.get(p.concept, 9))
        best = None
        tried = 0
        for p in setups:
            if tried >= self.cfg.eval_setup_k:
                break
            node = apply_play(s, p)
            if node is None:
                continue
            tried += 1
            val = expected(node, self._reply)
            if best is None or val[0].ev > best[0].ev:
                best = val
        return best

    def _reply(self, s):
        """Defence's best reply (ball-side plays), then the one-turn evaluation."""
        if s.stage != 'play' or s.side != 'X':
            return self._one(s)
        gen = self._gen
        res = []
        for p in gen.plays(s, False, light=True)[:self.cfg.eval_reply_k + 2]:
            node = apply_play(s, p)
            if node is not None:
                res.append(expected(node, self._one))
        if not res:
            return self._one(s)
        res.sort(key=lambda vs: vs[0].ev)
        return res[0]

    def _terms(self, s, v):
        c = self.cfg
        if not (c.term_open_lane or c.term_zone) or s.stage != 'play' or s.side != 'O':
            return v
        extra = 0.0
        if c.term_zone and B.on_board(s.o[s.ball]) and B.zone(s.o[s.ball]) != '.':
            extra += c.term_zone
        if c.term_open_lane:
            mine = v[0].look
            best = mine
            for r in range(5):
                if r == s.ball or not B.on_board(s.o[r]):
```

```
                    continue
                pend = s.with_(pending=(s.ball, r))
                if any(intercept_status(pend, j) for j in range(5)):
                    continue
                best = max(best, self.carrier_look(s, r, min(2, s.tok_x), True)[0])
            extra += c.term_open_lane * max(0.0, best - mine)
        return tuple(Val(x.ev + extra, x.look, x.p30, x.pexp, x.fs) for x in v)

    def _vals(self, s, base, ev, look, p30, shooter, pexp, fs):
        out = []
        for t in self.targets:
            coach = 0.0
            tgt = s.target if t is None else t
            if s.ctx.coach_call and tgt == shooter:
                coach = RC.COACH_BONUS[band(int(round(look)))]
            total = 0.0 if (s.zeroed) else base + ev + coach
            out.append(Val(total, look, p30, pexp, fs))
        return tuple(out)


# ============================================================================
# plays (the candidate generator)
# ============================================================================

@dataclass(frozen=True)
class Play:
    concept: str
    actions: tuple

    def text(self):
        return ' then '.join(str(a) for a in self.actions if a.kind != 'end_turn') or str(self.actions[0])


def _contest_arrow(rules, q, sp):
    """The arrow a defender arriving on q with a Move sets to contest a shooter on sp: facing him, and
    under the step 3h geometries the wedge with the best contest category."""
    if rules.contest_geometry == 'current' and rules.arrow_mode == 'single':
        return B.dir_between(q, sp)
    bt = G.best_terms(rules, q, None, sp)
    return bt[4] if bt else B.dir_between(q, sp)


def _seq_actions(d, seq, arrow):
    """Turn a reach sequence into engine actions (the Move's arrow set to `arrow`)."""
    out = []
    for item in seq:
        kind, q = item[0], item[1]
        if kind == 'move':
            out.append(Action('move', 'X', d, q, arrow if arrow is not None else 0))
        elif len(item) > 2:                                  # a wedge Sprint's chosen other edge
            out.append(Action(kind, 'X', d, q, item[2]))
        else:
            out.append(Action(kind, 'X', d, q))
    return out


def quick_legal(s, a):
    """Cheap legality check for an action in state s."""
    if a.kind == 'end_turn':
        from .actions import inbound_held
        return s.turn_n >= 1 or s.stage == 'reaction' or (s.pending and s.side == 'X' and not inbound_held(s))
    if a.kind in MOVEMENT:
        if not can_move_kind(s, a.side, a.piece, a.kind):
            return False
        start = (s.o if a.side == 'O' else s.x)[a.piece]
        if a.kind == 'move' and a.dest == start:
            return a.side == 'X' and a.arrow != s.arrows[a.piece]
        return a.dest in movement_dests(s, a.side, a.piece, a.kind, s.ctx.rules)
    if a.kind == 'steal':
        cp = s.o[s.ball]
        return (s.tok_x > 0 and s.count('X', 'steal') < RC.LIMITS['X']['steal']
                and G.faces(s.ctx.rules, s.arrows[a.piece], s.x[a.piece], cp))
    return a in legal_actions(s)


def apply_play(s, play):
    """Apply a play's actions in order; returns a state or ChanceNode, or None if illegal."""
    node = s
    for i, a in enumerate(play.actions):
        if isinstance(node, ChanceNode) or node.stage == 'over':
            return None
        if i > 0 and (node.side != a.side or not quick_legal(node, a)):
            return None                  # the turn passed (e.g. its action limit), or the action is illegal
        node = apply(node, a)
    if not isinstance(node, ChanceNode) and node.stage in ('play', 'reaction') and node.side == s.side \
            and node.turn_n >= 1 and node.stage == s.stage:
        node = apply(node, Action('end_turn', s.side))
    return node


class Generator:
    def __init__(self, cfg, ev):
        self.cfg = cfg
        self.ev = ev

    def plays(self, s, shallow, rng=None, light=False):
        if s.stage == 'target':
```

```
            out = [Play('target', (a,)) for a in legal_actions(s)]
        elif s.stage in ('pick', 'contest', 'forced'):
            out = [Play(s.stage, (a,)) for a in legal_actions(s)]
        elif s.stage == 'reaction':
            out = self.reaction_plays(s)
        elif s.side == 'O':
            out = self.offence_plays(s)
        else:
            out = self.defence_plays(s, light)
        if shallow and self.cfg.top_k_raw > 0 and s.stage in ('play', 'reaction'):
            out += self.raw_plays(s, out)
        seen, uniq = set(), []
        for p in out:
            if p.actions not in seen:
                seen.add(p.actions)
                uniq.append(p)
        if rng is not None:
            rng.shuffle(uniq)
        return uniq

    def _value_after(self, s, play):
        node = apply_play(s, play)
        if node is None:
            return None
        return expected(node, self.ev)[0].ev

    def raw_plays(self, s, have):
        firsts = {p.actions[0] for p in have}
        scored = []
        maximise = s.side == 'O' and s.stage != 'reaction'
        for a in legal_actions(s):
            if a in firsts:
                continue
            p = Play('raw', (a,))
            v = self._value_after(s, p)
            if v is not None:
                scored.append((v if maximise else -v, p))
        scored.sort(key=lambda x: -x[0])
        return [p for _, p in scored[:self.cfg.top_k_raw]]

    # ---- offence ----
… lines 695–761: printed in the offence document …
    # ---- defence ----
… lines 764–922: printed in the defence document …

… lines 924–946: printed in the defence document …

… lines 948–975: printed in the defence document …
def expected(node, fn):
    """Expected Val tuple of a node under fn (state -> Val tuple)."""
    if isinstance(node, ChanceNode):
        return mix([(float(p), expected(sub, fn)) for p, _, sub in node.outcomes])
    return fn(node)


# ============================================================================
# exact solver
# ============================================================================
… lines 989–990: printed in the appendix …

… lines 993–1002: printed in the appendix …
REACH_BOUND = 1 + RC.MOVE_STRAIGHT + 2 + 2   # adjacent square + longest Sprint (3 + QS 2) + margin = 8 (>= 6 needed)
… lines 1008–1038: printed in the appendix …

… lines 1041–1083: printed in the appendix …
# ============================================================================
# guided search
# ============================================================================
… lines 1090–1094: printed in the appendix …

… lines 1097–1261: printed in the appendix …
```

playmaker_engine/solver_config.py

Shared: every width and weight the evaluation uses.

```
"""Every limit, width, depth, weight and verdict threshold the solver uses."""
from dataclasses import dataclass, replace


@dataclass(frozen=True)
class SolverConfig:
    # ---- exact endgame solver ----
    endgame_tokens: int = 2          # N: positions in the final phase with <= N tokens per side go exact
    exact_node_cap: int = 4000       # give up on an exact solve after this many applies (fall back to guided)
    # ---- candidate generator ----
    top_k_raw: int = 8               # K: best raw actions (by evaluation) added to the concept plays
    raw_depth: int = 1               # add raw actions only this many plies from the root
    drive_k: int = 3                 # drive / drive-and-shoot squares per node
    relocate_k: int = 2              # off-ball relocations per node
    closeout_k: int = 4              # closeouts per node (outside the reaction to a shot)
    combo_k: int = 3                 # defence two-action combos built from the top single plays
    contain_k: int = 2               # the carrier's best step-then-shoot squares the defence plays to contain
    # ---- guided search ----
```

```
    beam: int = 6                    # plays kept per node below the root (ordered by evaluation)
    root_beam: int = 40              # plays kept at the root
    max_depth: int = 12              # iterative deepening limit, in turns
    # ---- evaluation ----
    eval_turns: int = 2              # the evaluation looks over the offence's next N turns (1 or 2)
    eval_setup_k: int = 4            # offence setup plays (pass / relocate / drive...) tried in turn 1
    eval_reply_k: int = 4            # defence replies tried to each setup
    # small positional terms (kept only if they improve agreement with the exact solver; see report)
    term_open_lane: float = 0.0      # per point of the best open-lane receiver's look above the carrier's
    term_zone: float = 0.0           # bonus if the carrier stands on a shooting zone (not plain wood)
    quiescence: bool = True          # a leaf where the defence is to move gets one more defence turn
    quiescence_beam: int = 12        # defence plays kept in that extra turn (ball-side plays only, no combos)
    kick_weight: float = 0.0         # value of the best teammate look (0 = carrier only); tested in the report
    inbound_kick_weight: float = 1.0 # during an inbound the ball must go in: use the best receiver's look
    # ---- budgets and confidence ----
    budgets_min: tuple = (2.0, 10.0) # time budgets per card, minutes
    seeds: int = 5
    unstable_spread: float = 2.0     # value spread (points) above which a card is unstable
    # ---- verdicts ----
    gift_p30: float = 0.75
    wall_p30: float = 0.15
    wall_look: float = 20.0
    plan_share: float = 0.90         # a plan counts if it reaches this share of the best value
    goal: int = 30


DEFAULT = SolverConfig()


def with_config(**kw):
    return replace(DEFAULT, **kw)
```

computer_levels.json (shared keys)

Thinking time, level names.

```
{
  "_about": "How the computer chooses among legal actions (step 3e). Levels are presets of the levers below. No level gets hidden information, extra tokens or the Coach's Call target. Edit freely: computer.py reads everything from here.",
  "budget_s": {
    "server": 2.0,
    "browser": 2.0
  },
  "_budget_about": "Thinking time per decision (raised in step 3f: the designer prefers a slower, better choice). The browser build (Python compiled to the web) runs several times slower; the best choice found so far is used when time is up.",
  "names": {
    "rookie": [
      "Rookie",
      "makes beginner mistakes you can learn to spot"
    ],
    "starter": [
      "Starter",
      "solid, with one habit to find"
    ],
    "allstar": [
      "All-Star",
      "plays the full read"
    ]
  },
  "habit_words": {
    "always_force_middle": "always pointed the on-ball arrow at the baseline, shading drives to the middle",
    "nearest_closes_out": "always closed out with the nearest defender",
    "never_leaves_shooter": "never left a ▲ three-point shooter, even when the look was poor",
    "ball_watcher": "turned every help defender's arrow to the ball",
    "jumps_first_pass": "tried to intercept every pass it could",
    "settles_early": "lowered its shot bar a phase early, taking looks it could have waited on",
    "settles_for_first_shot": "took the first look above a low bar",
    "never_screens": "never set a screen",
    "ignores_mismatch": "passed by position order, not by matchup",
    "star_only": "always worked toward its highest-rated scorer",
    "forces_passes": "passed to its primary target whenever it could, however well the lane was guarded"
  },
  "concept_words": {
    "reaction": "didn't count the reaction to a declared shot",
    "reserve": "didn't keep a reaction in hand",
    "lookahead": "looked only at the shot on the board now",
    "help": "didn't send help off the ball",
    "walls": "didn't wall drives with the on-ball arrow",
    "lanes": "didn't point arrows across passing lanes",
    "plan": "didn't build toward a target",
    "patience": "didn't weigh waiting for a better shot",
    "drain": "didn't work to drain the defence's reactions",
    "risk": "ignored turnover risk"
  },
  "_tuned": "Head-to-head calibration (step 3e revised): see README 'How the computer plays' for every change and why."
}
```

Part 2. The offence

playmaker_engine/computer.py

The offence: targets and plan, the shot bars, passing judgement (step 3f), advantage plays (step 3g), loops and memory.

```
def pass_risk(state, passer, receiver):
    """A pass judged after the defence's reply. A Pass ends the offence's turn and the defence answers
```

```
    before it resolves, so the lane is clean only if no defender can legally get an Intercept on it in
    that turn: directly (on the line) or by an approach (a Move, Quick Step or Sprint onto the line by a
    defender whose arrow already crosses it, then Intercept), with the defence's tokens, unused Moves,
    locks and the intercept limit as the rules have them. Computed from the legal actions, so it is the
    rules' own answer. Returns None if the pass isn't legal, else a dict: clean, risk (the best
    interception chance any defender can reach: 1 - P(d20 <= Pass - Intercept)), by (that defender),
    line (the defence's actions for it), pending (the state with the ball in the air)."""
    from .actions import is_approach
    from .scoring import p_success
    key = (state, id(state.ctx), passer, receiver)         # positions compare equal across contexts
    hit = _PASS_CACHE.get(key)
    if hit is not None and hit['ctx'] is state.ctx:
        return hit
    a = Action('pass', 'O', passer, target=receiver)
    if state.stage != 'play' or state.side != 'O' or state.ball != passer or a not in legal_actions(state):
        return None
    pend = apply(state, a)
    out = dict(clean=True, risk=0.0, by=None, line=(), pending=pend, ctx=state.ctx)
    if not isinstance(pend, ChanceNode) and pend.pending:
        from .actions import inbound_held, intercept_status, movement_actions
        pr = pend.ctx.off[passer].r['pass']
        if pend.inbound:                          # inbound passes (OQ-35): ask the legal actions directly
            cands = []
            for d in legal_actions(pend):
                if d.kind == 'intercept':
                    cands.append((d,))
                elif d.kind in MOVEMENT and is_approach(pend, d):
                    nxt = apply(pend, d)
                    icpt = Action('intercept', 'X', d.piece)
                    if not isinstance(nxt, ChanceNode) and icpt in legal_actions(nxt):
                        cands.append((d, icpt))
        else:                                     # the same answer as the legal actions, computed directly
            cands = []
            can = pend.count('X', 'intercept') < RC.LIMITS['X']['intercept'] and pend.tok_x > 0
            status = [intercept_status(pend, j) for j in range(5)] if can else [None] * 5
            for j in range(5):
                if status[j] == 'direct':
                    cands.append((Action('intercept', 'X', j),))
            if any(st == 'approach' for st in status):
                from .actions import can_move_kind, movement_dests
                a_sq, b_sq = pend.o[passer], pend.o[receiver]
                tol = pend.ctx.rules.on_line_tolerance
                for j in range(5):
                    if status[j] != 'approach':
                        continue
                    done = False
                    for kind in ('move', 'qs', 'sprint'):          # only this defender, only onto the line
                        if done or not can_move_kind(pend, 'X', j, kind):
                            continue
                        for q in movement_dests(pend, 'X', j, kind, pend.ctx.rules):
                            if not B.on_segment(q, a_sq, b_sq, tol):
                                continue
                            d = Action(kind, 'X', j, q, pend.arrows[j]) if kind == 'move' else Action(kind, 'X', j, q)
                            if not is_approach(pend, d):
                                continue
                            nxt = apply(pend, d)
                            if not isinstance(nxt, ChanceNode) and nxt.approach == j and nxt.tok_x > 0 and \
                                    nxt.count('X', 'intercept') < RC.LIMITS['X']['intercept']:
                                cands.append((d, Action('intercept', 'X', j)))
                                done = True                        # one approach is enough: same roll
                                break
        for line in cands:
            j = line[-1].piece
            r = 1.0 - float(p_success(pr - pend.ctx.dfn[j].r['intercept'], pend.ctx.rules))
            if out['clean'] or r > out['risk'] or (r == out['risk'] and len(line) < len(out['line'])):
                out.update(clean=False, risk=r, by=j, line=line)
    if len(_PASS_CACHE) > 20000:
        _PASS_CACHE.clear()
    _PASS_CACHE[key] = out
    return out
```

… lines 218–219: printed in the shared core (Part 1) …

```
def pass_outcomes(info):
    """The two positions a pass can lead to, by the rules: caught (the defence's interception attempt
    failed, or there was none) and picked off (a turnover; the ball stays with the passer and play goes
    on, OQ-1). Picked off is None for a clean lane."""
    from .resolve import complete_pass, force
    pend = info['pending']
    if isinstance(pend, ChanceNode) or not pend.pending:
        return pend, None
    if info['clean']:
        return complete_pass(pend), None          # it lands at the start of the defence's turn
    node = pend
    for d in info['line']:
        node = apply(node, d)
    return force(node, 'success'), force(node, 'failure')
```

… lines 234–455: printed in the shared core (Part 1) …

```
    @staticmethod
    def _spot(state):
        return (state.o, state.x, state.arrows, state.ball)
```

… lines 459–459: printed in the shared core (Part 1) …

```
    def _remember(self, state):
        if self._last_pass is not None and state.tos > self._last_pass[2]:
            self.picked_off.add(self._last_pass[:2])
        self._last_pass = None
        k = (state.o, state.ball)
        self.visited[k] = self.visited.get(k, 0) + 1
```

… lines 466–466: printed in the shared core (Part 1) …

```
    def _note_pass(self, state, a):
        if self.side == 'O' and a.kind == 'pass':
            self._last_pass = (self._spot(state), a.target, state.tos)
```
… lines 470–470: printed in the shared core (Part 1) …
```
    def _loop_penalty(self, view, node):
        """No progress: a play that ends where the offence already stood earlier this possession (its five
        squares and the ball) loses this much per earlier visit (at most three)."""
        w = self.cfg.get('loop_penalty', LEVELS['offence'].get('loop_penalty', 0))
        if not w or isinstance(node, ChanceNode):
            return 0.0
        k = (node.o, node.pending[1] if node.pending else node.ball)
        n = self.visited.get(k, 0) - (1 if k == (view.o, view.ball) else 0)
        return w * min(3, max(0, n))
```
… lines 480–716: printed in the shared core (Part 1) and the defence document …
```
    def targets(self, view):
        """Primary and secondary targets: the two largest edges, each a player's ▲ shot type against the
        defender likely to contest it (the nearest), read at the level's card reading."""
        ctx = view.ctx
        three_only = ctx.three_only
        edges = []
        for i, c in enumerate(ctx.off):
            if not B.on_board(view.o[i]) and i != view.ball:
                continue
            near = min(range(5), key=lambda j: B.cheb(view.x[j], view.o[i]))
            for t in ('layup', 'dunk', 'post', 'floater', 'mid', 'long', 'three'):
                if c.bands['shot_' + t] != 'H' or (three_only and t != 'three'):
                    continue
                pts = 3 if t == 'three' else 2
                edges.append((pts * (c.shot(t) - ctx.dfn[near].con(t)), i, t))
        edges.sort(key=lambda e: (-e[0], e[1]))
        if 'star_only' in self.habits:
            star = max(range(5), key=lambda i: (max(ctx.off[i].r[k] for k in ctx.off[i].r if k.startswith('shot_')), -i))
            edges = [e for e in edges if e[1] == star] + [e for e in edges if e[1] != star]
            self._fire('star_only')
        out = []
        for e in edges:
            if e[1] not in [x[1] for x in out]:
                out.append(e)
        if ctx.coach_call and view.target >= 0:
            out = [x for x in out if x[1] == view.target] + [x for x in out if x[1] != view.target]
            if not out or out[0][1] != view.target:
                out.insert(0, (0, view.target, 'three' if three_only else 'any'))
        return [dict(o=i, shot=t, edge=v) for v, i, t in out[:2]]
```
… lines 746–746: printed in the shared core (Part 1) …
```
    def _ensure_plan(self, view):
        c = self.cfg['concepts']
        if not c.get('plan', True) and 'star_only' not in self.habits and not view.ctx.coach_call:
            return
        reset = self.plan is None or (view.phase != self.plan['phase'] and view.inbound == 0 and self.plan.get('inbound'))
        if self.plan is not None and not reset:
            prim = self.plan['targets'][0]['o'] if self.plan['targets'] else None
            if prim is not None and prim != view.ball:
                near = min(range(5), key=lambda j: B.cheb(view.x[j], view.o[prim]))
                t = self.plan['targets'][0]['shot']
                switched = t in ('layup', 'dunk', 'post', 'floater', 'mid', 'long', 'three') and \
                    view.ctx.dfn[near].bands['con_' + t] == 'H' and near != self.plan['near']
                if switched and not view.ctx.coach_call:
                    reset = True
                    self.notes.append(dict(event='replan', why=f'a ▲ {t} contester (X{near + 1}) took O{prim + 1}'))
        if reset:
            tg = self.targets(view)
            near = min(range(5), key=lambda j: B.cheb(view.x[j], view.o[tg[0]['o']])) if tg else None
            before = self.plan['type'] if self.plan else None
            kind = self._plan_kind(view, tg)
            self.plan = dict(targets=tg, phase=view.phase, inbound=bool(view.inbound), near=near, type=kind)
            self.notes.append(dict(event='plan', targets=tg))
```
… lines 769–769: printed in the shared core (Part 1) …
```
    def _plan_bonus(self, view, node_state):
        if not self.plan or not self.plan['targets'] or isinstance(node_state, ChanceNode):
            return 0.0
        w = LEVELS['offence']['plan_weight'] * (3 if 'star_only' in self.habits else 1)
        tg = self.plan['targets'][0]
        s = node_state
        bonus = 0.0
        if s.ball == tg['o']:
            bonus += w
        z = B.zone(s.o[tg['o']]) if B.on_board(s.o[tg['o']]) else '.'
        if tg['shot'] in RC.ZONE_SHOTS.get(z, ()):
            bonus += w
        return bonus
```
… lines 783–783: printed in the shared core (Part 1) …
```
    def _attack(self, view, acts):
        from .solver import Play, apply_play
        if view.stage == 'target':
            tg = self.targets(view)
            want = tg[0]['o'] if tg else 0
            return Play('target', (next((a for a in acts if a.target == want), acts[0]),))
        if view.stage in ('pick', 'forced'):
            scored = [(self._node_value(apply(view, a), lambda s: self.ev(s)[0].ev), Play(view.stage, (a,))) for a in acts]
            return self._pick(scored, True)
        self._ensure_plan(view)
        c = self.cfg['concepts']
        if self.cfg['lookahead'] == 'solver':
            p = self._search(view)
            if p is not None and self._patient(view, p, None):
                return p
```

```
        plays = self._with_end(view, acts, self.gen.plays(view, shallow=False))
        plays = self._filter_plays(view, plays)
        if 'forces_passes' in self.habits and self.plan and self.plan['targets']:
            t = self.plan['targets'][0]['o']
            fp = next((a for a in acts if a.kind == 'pass' and a.target == t), None)
            if fp is not None:                              # to the primary target whenever legal, whatever the risk
                self._fire('forces_passes')
                return Play('habit', (fp,))
        if self.cfg.get('plays'):
            chosen = self._advantage_attack(view, acts)
            if chosen is not None:
                return chosen
        self._plan_note(view, None, None)          # the fallback: no advantage play scored above zero
        fn = self._value_fn()
        late = self._late()
        if late:
            fn = self._immediate
        stop = self._stop()
        scored, raw = [], {}
        for p in plays:
            if stop is not None and scored and time.time() > stop:
                break
            node = apply_play(view, p)
            if node is None:
                continue
            v = self._node_value(node, fn) if late else self._play_value(view, p, node, fn)
            raw[p] = v                                      # the bar reads the shot's value, not the plan bonus
            if c.get('plan', True) or 'star_only' in self.habits:
                v += self._plan_bonus(view, node)
            if not any(a.kind == 'shot' for a in p.actions):
                v -= self._loop_penalty(view, node)
            scored.append((v, p))
        if not scored:
            return None
        shots = [(v, p) for v, p in scored if any(a.kind == 'shot' for a in p.actions)]
        rest = [(v, p) for v, p in scored if not any(a.kind == 'shot' for a in p.actions)]
        best = self._pick(scored, True)
        if best is None or not shots or not rest:
            return best
        # 7.3 with a bar: a shot goes up only when it clears the level's bar for the time left (higher for
        # a look a defender can still contest than for an open one) and is worth at least what continuing
        # is estimated to be worth (a tie goes to the shot). The bars drop each phase, so a defence that
        # keeps the good looks covered forces a lesser one late. On the last chance there is no bar.
        # The bar reads the look itself, not points already banked this possession (turnovers, the
        # steal-foul bonus). From the level's shoot_by clock on, the best shot goes up whatever it is.
        last = self._last_chance(view)
        banked = RC.STEAL_FOUL_BONUS * view.bonus5 + RC.TURNOVER_POINTS * view.tos
        is_shot = lambda p: any(a.kind == 'shot' for a in p.actions)
        if view.clock <= self.cfg.get('shoot_by', 0) and not last:
            return max(shots, key=lambda vp: vp[0])[1]
        cleared = [(v, p) for v, p in shots if last or raw[p] - banked >= self._bar(view, self._contested(view, p))]
        cv = max(v for v, p in rest)
        if cleared and max(v for v, p in cleared) >= cv - 1e-9:
            sv, sp = max(cleared, key=lambda vp: vp[0])
            if not last and 'settles_early' in self.habits and \
                    raw[sp] - banked < self._bar(view, self._contested(view, sp), early=False):
                self._fire('settles_early')
            best = sp
        elif is_shot(best):
            best = self._pick(rest, True)
        return best
```

… lines 860–861: printed in the shared core (Part 1) …

```
    _ALT = {'relocate': 'receiver', 'drive': 'passer', 'dribble': 'passer', 'hesitate': 'passer',
            'screen': 'screen', 'seal': 'screen'}
```

… lines 864–864: printed in the shared core (Part 1) …

```
    def _filter_plays(self, view, plays):
        """The picked-off memory and the habits that strike plays out (never screens, ignores the mismatch,
        star only)."""
        if self.picked_off:
            spot = self._spot(view)
            kept = [p for p in plays if not any(a.kind == 'pass' and (spot, a.target) in self.picked_off
                                                for a in p.actions)]
            plays = kept or plays
        if 'never_screens' in self.habits:
            kept = [p for p in plays if not any(a.kind == 'screen' for a in p.actions)]
            if len(kept) < len(plays):
                self._fire('never_screens')
            plays = kept or plays
        if 'ignores_mismatch' in self.habits:
            order = [i for i in list(range(view.ball + 1, 5)) + list(range(view.ball)) if B.on_board(view.o[i])]
            nxt = order[0] if order else None
            kept = [p for p in plays if not any(a.kind == 'pass' and a.target != nxt for a in p.actions)]
            if len(kept) < len(plays):
                self._fire('ignores_mismatch')
            plays = kept or plays
        if 'star_only' in self.habits and self.plan and self.plan['targets']:
            star = self.plan['targets'][0]['o']
            kept = [p for p in plays if not any(a.kind == 'pass' and a.target != star for a in p.actions)]
            plays = kept or plays
        return plays
```

… lines 890–890: printed in the shared core (Part 1) …

```
    def _play_value(self, view, p, node, fn):
        """A play's value. A pass is valued by what the catch gives against the risk after the defence's
        reply (pass_risk); the alternatives this level weighs (moving the receiver, moving the passer or
        dribbling, a screen or seal) are valued with the offence's next turn able to pass, so a cut that
        opens a clean lane is worth that lane. Everything else as before."""
        alts = self.cfg.get('pass_alternatives', [])
```

```
        if p.actions and p.actions[-1].kind == 'pass' and not view.inbound:
            v = self._pass_play_value(view, p, follow='two_pass' in alts)
            if v is not None:
                return v
        if self._ALT.get(p.concept) in alts:
            return self._node_value(node, lambda st: self._after_value(st, True))
        return self._node_value(node, fn)
```

… lines 904–904: printed in the shared core (Part 1) …

```
    @staticmethod
    def _banked(s):
        return RC.STEAL_FOUL_BONUS * s.bonus5 + RC.TURNOVER_POINTS * s.tos
```

… lines 908–908: printed in the shared core (Part 1) …

```
    def _pick_off_value(self, pre):
        """Step 3f, section 2: a pass is valued (1 - risk) x the catch - risk x 5. A pick-off counts as the
        -5 alone (scaled by the level's pass_risk_tolerance: 1.0 honest, below 1.0 reckless), on top of
        what is already banked, not as the position it leaves: under OQ-1 the turnover plays on, and that
        position is often worth more to the offence than the catch (the interceptor has left his man and
        spent two tokens), which would make every contested pass nearly free."""
        return self._banked(pre) + RC.TURNOVER_POINTS * self.cfg.get('pass_risk_tolerance', 1.0)
```

… lines 916–916: printed in the shared core (Part 1) …

```
    def _pass_play_value(self, view, p, follow):
        """(1 - risk) x the position after the catch (valued after the defence's reply) + risk x the
        pick-off (see _pick_off_value)."""
        pre = view
        for a in p.actions[:-1]:
            pre = apply(pre, a)
            if isinstance(pre, ChanceNode) or pre.stage != 'play' or pre.side != 'O':
                return None
        a = p.actions[-1]
        info = pass_risk(pre, a.piece, a.target)
        if info is None:
            return None
        caught, lost = pass_outcomes(info)
        cv = self._node_value(caught, lambda st: self._after_value(st, follow))
        if lost is None:
            return cv
        return (1 - info['risk']) * cv + info['risk'] * self._pick_off_value(pre)
```

… lines 934–934: printed in the shared core (Part 1) …

```
    def _after_value(self, st, follow):
        """A position after the offence's play: the defence's best reply (its ball-side plays), then the
        offence's next action; with follow, that next action may be a pass (valued the same way)."""
        from .solver import apply_play
        if isinstance(st, ChanceNode):
            return self._node_value(st, lambda x: self._after_value(x, follow))
        if st.stage != 'play':
            return self.ev(st)[0].ev
        if st.side == 'O':
            return self._o_next(st, follow)
        res = []
        stop = self._stop()
        # the defence's full reply set (recovers, switches, contains, combos too): a reply limited to the
        # ball side would never cover the teammate the next pass goes to, and overrate every alternative
        for q in self._with_end(st, legal_actions(st), self.gen.plays(st, False, light=False)):
            if stop is not None and res and time.time() > stop:
                break                                   # out of thinking time: the replies read so far
            n = apply_play(st, q)
            if n is not None:
                res.append(self._node_value(n, lambda x: self._o_next(x, follow) if (
                    not isinstance(x, ChanceNode) and x.stage == 'play' and x.side == 'O') else self.ev(x)[0].ev))
        return min(res) if res else self._o_next(st, follow)
```

… lines 957–957: printed in the shared core (Part 1) …

```
    def _o_next(self, s, follow):
        """The offence to act: its best action now (the evaluation), or with follow a pass to any
        teammate, judged after the defence's reply like the first one (the extra pass costs a turn: the
        defence answers it before the catch is valued)."""
        v = self.ev(s)[0].ev
        if not follow or s.pending or s.inbound or s.tok_o < 1:
            return v
        stop = self._stop()
        if stop is not None and time.time() > stop:
            return v                                    # out of thinking time: no follow-up pass read
        memo = self.__dict__.setdefault('_next_memo', {})
        if s in memo and memo[s][0] is s.ctx:
            return memo[s][1]
        budget_x = reaction_budget(s)
        reply = lambda t: self.ev._reply(t)[0].ev if (not isinstance(t, ChanceNode) and t.stage == 'play'
                                                      and t.side == 'X') else self.ev(t)[0].ev
        best = v
        for r in range(5):
            if r == s.ball or not B.on_board(s.o[r]):
                continue
            if self.ev.carrier_look(s, r, budget_x, True)[0] + RC.STEAL_FOUL_BONUS * s.bonus5 \
                    + RC.TURNOVER_POINTS * s.tos <= best:
                continue                                # even an open catch there can't beat what it has
            info = pass_risk(s, s.ball, r)
            if info is None:
                continue
            caught, lost = pass_outcomes(info)
            cv = self._node_value(caught, reply)
            if lost is not None:
                cv = (1 - info['risk']) * cv + info['risk'] * self._pick_off_value(s)
            best = max(best, cv)
        if len(memo) > 20000:
            memo.clear()
        memo[s] = (s.ctx, best)
        return best
```

```
… lines 993–994: printed in the shared core (Part 1) …
    PASS_KINDS = ('mismatch', 'numbers', 'beaten', 'bad_closeout', 'spent')
… lines 996–996: printed in the shared core (Part 1) …
    def _maps(self, s):
        return self.ev.reach(s, reaction_budget(s))
… lines 999–999: printed in the shared core (Part 1) …
    def _advs(self, s, prev=None):
        from .advantage import advantages
        return advantages(s, prev=prev, maps=self._maps(s))
… lines 1003–1003: printed in the shared core (Part 1) …
    def _note_handover(self, state, a):
        """The board as the offence hands the turn to the defence (help_pulled compares against it)."""
        if self.side != 'O' or state.stage != 'play' or state.side != 'O':
            return
        nxt = apply(state, a)
        if not isinstance(nxt, ChanceNode) and nxt.stage == 'play' and nxt.side == 'X' and not nxt.pending:
            self._handover = nxt
        elif isinstance(nxt, ChanceNode) or nxt.pending:
            self._handover = None
… lines 1013–1013: printed in the shared core (Part 1) …
    def _prev(self, view):
        h = self._handover
        if h is not None and h.ctx is view.ctx and h.o == view.o:
            return h
        return None
… lines 1019–1019: printed in the shared core (Part 1) …
    def _exploit(self, s, prev, skip=None):
        """Offence turn 2: the best finish of an advantage on the board, by the carrier, or by a pass to
        the attacker who has it (judged by pass_risk; help_pulled only for a level that moves advantages).
        Returns (value, advantage) or (0, None) when there is none."""
        known = self.cfg['plays']
        kinds = self.PASS_KINDS + (('help_pulled',) if 'move_advantage' in known else ())
        tol = self.cfg.get('pass_risk_tolerance', 1.0)
        best, which = 0.0, None
        for a in self._advs(s, prev):
            if skip and (a['kind'], a['attacker']) in skip:
                continue                          # it was there already: this play didn't create it
            if a['attacker'] == s.ball:
                v = a['value']
            elif 'pass' in known and a['kind'] in kinds:
                if s.stage == 'play' and s.side == 'O':
                    info = pass_risk(s, s.ball, a['attacker'])
                    if info is None:
                        continue
                    v = (1 - info['risk']) * a['value'] + info['risk'] * RC.TURNOVER_POINTS * tol
                else:
                    v = a['value']                # the offence passes next turn
            else:
                continue
            if v > best:
                best, which = v, a
        return best, which
… lines 1046–1046: printed in the shared core (Part 1) …
    def _tempo(self, view, s):
        """The clock spent, valued by the patience curve: how far the open-look bar falls meanwhile."""
        if isinstance(s, ChanceNode) or s.stage == 'over':
            return 0.0
        return max(0.0, self._bar(view, False, early=False) - self._bar(s, False, early=False))
… lines 1052–1052: printed in the shared core (Part 1) …
    def _adv_value(self, view, node, depth):
        """A play's score: the best advantage finish reached, plus points banked on the way (a turnover
        -5, a steal-foul bonus), less the tempo. depth 1 = create only (the board after the play);
        depth 2 = create, the defence's best reply, then exploit."""
        from .solver import apply_play
        if isinstance(node, ChanceNode):
            return sum(float(p) * self._adv_value(view, sub, depth) for p, _, sub in node.outcomes)
        if node.stage == 'over':
            return self.ev(node)[0].ev - self._banked(view)
        if node.stage != 'play':
            return self.ev(node)[0].ev - self._banked(view)

        before = None
        if depth < 2:                         # create only: an advantage counts if the play created it
            before = {(a['kind'], a['attacker']) for a in self._advs(view, self._prev(view))}

        def score(st, prev):
            if isinstance(st, ChanceNode):
                return sum(float(p) * score(sub, prev) for p, _, sub in st.outcomes)
            if st.stage != 'play':
                return self.ev(st)[0].ev - self._banked(view)
            return self._exploit(st, prev, before)[0] + self._banked(st) - self._banked(view) - self._tempo(view, st)
        if depth < 2 or node.side == 'O':
            return score(node, None)
        res = []
        stop = self._stop()
        for q in self._with_end(node, legal_actions(node), self.gen.plays(node, False, light=False)):
            n = apply_play(node, q)
            if n is not None:
                res.append(score(n, node))
            if stop is not None and res and time.time() > stop:
                break
        return min(res) if res else score(node, None)
… lines 1086–1086: printed in the shared core (Part 1) …
    def _adv_plays(self, view, acts):
        """Advantage plays, one offence turn each, by type (the level's plays only)."""
```

```
        from .solver import Play
        known = self.cfg['plays']
        c = view.ball
        two = view.tok_o >= 2
        gen = self.gen.offence_plays(view)
        out = {k: [] for k in ('shoot', 'dribble', 'screen', 'speed', 'seal', 'cut', 'pass')}
        out['shoot'] = [p for p in gen if p.actions and p.actions[-1].kind == 'shot']
        out['dribble'] = [Play('dribble attack', (a,)) for a in acts if a.kind == 'dribble']
        maps = None
        for a in acts:
            if a.kind in ('qs', 'sprint'):              # speed burst: to a square nobody can cleanly contest
                n = apply(view, a)
                if isinstance(n, ChanceNode) or n.stage != 'play':
                    continue
                from .advantage import looks
                lk = looks(n, a.piece, self._maps(n))
                if lk and not any(e['clean'] for e in lk[0][2]):
                    out['speed'].append(Play('speed burst', (a,)))
                    if a.piece == c and n.side == 'O':
                        sh = next((b for b in legal_actions(n) if b.kind == 'shot'), None)
                        if sh is not None:
                            out['shoot'].append(Play('speed+shot', (a, sh)))
            if a.kind in ('move', 'qs') and a.piece != c and B.on_board(a.dest):
                out['cut'].append(Play('cut', (a,)))
        # screens: every screener and defender pair the screener can reach this turn
        for p_ in range(5):
            if not B.on_board(view.o[p_]) or view.is_screening(p_):
                continue
            for a in acts:
                if a.kind == 'screen' and a.piece == p_:
                    out['screen'].append(Play('screen', (a,)))
            if not two:
                continue
            for a in acts:
                if a.kind in ('move', 'qs') and a.piece == p_ and any(B.adjacent(a.dest, xq) for xq in view.x):
                    n = apply(view, a)
                    if isinstance(n, ChanceNode) or n.stage != 'play' or n.side != 'O':
                        continue
                    for b in legal_actions(n):
                        if b.kind == 'screen' and b.piece == p_:
                            out['screen'].append(Play('screen', (a, b)))
        # post seal: a ▲ post scorer seals a ▼ post defender (the generator's seal plays)
        for p in gen:
            if p.concept == 'seal':
                scr = p.actions[-1]
                tq = B.add(view.o[scr.piece] if len(p.actions) == 1 else p.actions[0].dest, B.DIRS[scr.target])
                j = view.x.index(tq) if tq in view.x else None
                if j is not None and view.ctx.off[scr.piece].bands['shot_post'] == 'H' and \
                        view.ctx.dfn[j].bands['con_post'] == 'L':
                    out['seal'].append(p)
        # passes: only to move an advantage to a better attacker, or finish one
        kinds = self.PASS_KINDS + (('help_pulled',) if 'move_advantage' in known else ())
        advs = self._advs(view, self._prev(view))
        mine = max((a['value'] for a in advs if a['attacker'] == c), default=0.0)
        seen = set()
        for a in advs:
            r = a['attacker']
            if r == c or r in seen or a['kind'] not in kinds or a['value'] <= mine:
                continue
            pa = Action('pass', 'O', c, target=r)
            if pa in acts:
                seen.add(r)
                out['pass'].append(Play('pass to ' + a['kind'], (pa,)))
        keep = {'shoot': 'shoot', 'dribble': 'dribble', 'pass': 'pass', 'screen': 'screen', 'speed': 'speed',
                'seal': 'seal', 'cut': 'cut'}
        return {k: v for k, v in out.items() if keep[k] in known or k == 'shoot'}
```

… lines 1155–1155: printed in the shared core (Part 1) …

```
    def _advantage_attack(self, view, acts):
        """Step 3g: the best advantage play. Shots by the shot bars (7.3); every other play by
        create (the play), reply (the defence's best answer), exploit (the best finish of an advantage),
        less turnover risk and tempo. None if no play scores above zero (the caller falls back to the raw
        actions)."""
        from .solver import Play, apply_play
        depth = self.cfg.get('advantage_depth', 2)
        cats = self._adv_plays(view, acts)
        for k in cats:
            cats[k] = self._filter_plays(view, cats[k])
        stop = self._stop()
        shots = []
        for p in cats.pop('shoot', []):
            node = apply_play(view, p)
            if node is not None:
                shots.append((self._node_value(node, self._value_fn()), p))
        scored = []
        tol = self.cfg.get('pass_risk_tolerance', 1.0)
        # the one-turn estimates get 40% of the thinking time; the full reads (top three a type) the rest
        est_stop = None if stop is None else time.time() + 0.4 * max(0.0, stop - time.time())
        for k, plays in cats.items():
            est = []
            for p in plays:
                if est_stop is not None and est and time.time() > est_stop:
                    break
                node = apply_play(view, p)
                if node is None:
                    continue
                if k == 'pass':
                    info = pass_risk(view, view.ball, p.actions[-1].target)
```

```
                    caught, lost = pass_outcomes(info)
                    one = lambda st: self._adv_value(view, st, 1)
                    full = lambda st: self._adv_value(view, st, depth)
                    v1 = (1 - info['risk']) * one(caught) + info['risk'] * RC.TURNOVER_POINTS * tol
                    est.append((v1, p, lambda c=caught, i=info, f=full: (1 - i['risk']) * f(c) + i['risk'] * RC.TURNOVER_POINTS * tol))
                else:
                    est.append((self._adv_value(view, node, 1), p, lambda n=node: self._adv_value(view, n, depth)))
            est.sort(key=lambda e: -e[0])
            for v1, p, full in est[:3]:                 # the top three by a one-turn estimate, then the full read
                if stop is not None and scored and time.time() > stop:
                    scored.append((v1, p, k))
                    continue
                v = full() if depth >= 2 else v1
                node = apply_play(view, p)
                scored.append((v - self._loop_penalty(view, node), p, k))
        banked = self._banked(view)
        best = max(scored, key=lambda e: e[0]) if scored else None
        cv = best[0] if best else 0.0
        last = self._last_chance(view)
        if shots:
            if view.clock <= self.cfg.get('shoot_by', 0) and not last:
                sp = max(shots, key=lambda vp: vp[0])[1]
                self._plan_note(view, sp, None)
                return sp
            cleared = [(v, p) for v, p in shots if last or v - banked >= self._bar(view, self._contested(view, p))]
            if cleared:
                sv, sp = max(cleared, key=lambda vp: vp[0])
                if sv - banked >= cv - 1e-9:
                    if not last and 'settles_early' in self.habits and \
                            sv - banked < self._bar(view, self._contested(view, sp), early=False):
                        self._fire('settles_early')
                    self._plan_note(view, sp, None)
                    return sp
        if best is None or best[0] <= 0:
            return None
        self._plan_note(view, best[1], best[2])
        return best[1]
```

⋯ lines 1223–1224: printed in the shared core (Part 1) ⋯

```
    def _play_words(self, view, p, kind):
        a = p.actions[-1]
        who = lambda i: f'O{i + 1}'
        if kind == 'screen':
            tq = B.add(p.actions[0].dest if len(p.actions) > 1 else view.o[a.piece], B.DIRS[a.target])
            j = view.x.index(tq) if tq in view.x else None
            man = min((i for i in range(5) if i != a.piece), key=lambda i: (B.cheb(view.x[j], view.o[i]), i)) \
                if j is not None else None
            if j is None:
                return f"Screen: {who(a.piece)} sets the T at {B.sq(tq)}"
            return f"Screen for a mismatch: {who(a.piece)} screens X{j + 1} off {who(man)}"
        if kind == 'dribble':
            return f"Dribble attack: {who(view.ball)} at X{a.target + 1}"
        if kind == 'speed':
            return f"Speed burst: {who(a.piece)} {'Sprints' if a.kind == 'sprint' else 'Quick Steps'} to {B.sq(a.dest)}"
        if kind == 'seal':
            return f"Post seal: {who(a.piece)} seals for the post entry"
        if kind == 'cut':
            return f"Cut: {who(a.piece)} to {B.sq(a.dest)}"
        if kind == 'pass':
            return f"Pass to {who(a.target)} for {p.concept.replace('pass to ', '').replace('_', ' ')}"
        return p.concept
```

⋯ lines 1247–1247: printed in the shared core (Part 1) ⋯

```
    def _plan_note(self, view, p, kind):
        """Keep the plan while its attacker still has an advantage; if help moved it, follow it (logged as
        'advantage moved'); a shot finishes it. Log plan changes only."""
        from .advantage import describe
        advs = self._advs(view, self._prev(view)) if view.stage == 'play' else []
        ap = self.aplan
        if ap is not None and ap['phase_ok'] and any(a['attacker'] == ap['attacker'] for a in advs):
            pass                                            # still working it
        elif ap is not None:
            moved = [a for a in advs if a['kind'] == 'help_pulled']
            if moved:
                a = moved[0]
                ap['attacker'] = a['attacker']
                self.notes.append(dict(event='advantage_moved', attacker=a['attacker'], helper=a['helper'],
                                       shot=a['shot'], text=f"Help came from X{a['helper'] + 1}; the advantage moved to O{a['attacker']
+ 1}"))
            elif ap.get('created'):
                self.notes.append(dict(event='advantage_gone', text=f"The defence took it away from O{ap['attacker'] + 1}"))
                self.aplan = ap = None
            else:
                ap['created'] = bool(advs)
        if p is None:
            return
        if any(x.kind == 'shot' for x in p.actions):
            if self.aplan is not None:
                sh = next(x for x in p.actions if x.kind == 'shot')
                self.notes.append(dict(event='finish', attacker=sh.piece, shot=sh.shot,
                                       text=f"Finished: O{sh.piece + 1} {sh.shot}"))
                self.aplan = None
            return
        if self.aplan is not None or kind is None:
            return
        from .solver import apply_play
        node = apply_play(view, p)
        if isinstance(node, ChanceNode):                    # a dribble or a screen: the advantage it is for
            node = next((sub for _, lab, sub in node.outcomes if lab == 'success'), node.outcomes[0][2])
```

```
        after = [] if node is None or isinstance(node, ChanceNode) or node.stage != 'play' else self._advs(node)
        target = after[0]['attacker'] if after else (p.actions[-1].target if p.actions[-1].kind == 'pass' else view.ball)
        text = self._play_words(view, p, kind)
        self.aplan = dict(kind=kind, attacker=target, text=text, phase_ok=True, created=bool(after))
        self.notes.append(dict(event='play', play=kind, attacker=target, text=text,
                               advantage=describe(after[0]) if after else None))
… lines 1289–1289: printed in the shared core (Part 1) …
    def _contested(self, view, play):
        """Could a defender still contest this shot after the reaction (as this level reads the reaction)?"""
        from .solver import apply_play
        node = apply_play(view, play)
        if node is None or isinstance(node, ChanceNode) or node.stage not in ('reaction', 'contest') or not node.shot:
            return False
        budget = reaction_budget(node) if node.stage == 'reaction' and self.cfg['concepts'].get('reaction', True) else 0
        return any(e['contest'] > 0 for e in reachable_contest(node, node.ball, node.shot, budget))
… lines 1298–1298: printed in the shared core (Part 1) …
    def _chances(self, view):
        """Clock left in seconds (a phase is two; its tokens reset each phase)."""
        return view.clock
… lines 1302–1302: printed in the shared core (Part 1) …
    def _last_chance(self, view):
        return self._chances(view) <= RC.PHASE_SECONDS
… lines 1305–1305: printed in the shared core (Part 1) …
    def _bar(self, view, contested, early=True):
        """The shot bar for the time left: 'open' for a look nobody can contest, 'contested' otherwise.
        The habit settles_early reads the bar one phase ahead."""
        t = self._chances(view)
        if early and 'settles_early' in self.habits and t > RC.PHASE_SECONDS:
            t -= RC.PHASE_SECONDS
        curve = {int(k): v for k, v in self.cfg['patience']['contested' if contested else 'open'].items()}
        below = [k for k in sorted(curve) if k <= t]
        return curve[below[-1]] if below else 0
… lines 1315–1315: printed in the shared core (Part 1) …
    def _patient(self, view, play, shot_value):
        """For the solver-search path: the shot clears the bar, or it's the last chance."""
        if not any(a.kind == 'shot' for a in play.actions) or shot_value is None or self._last_chance(view):
            return True
        banked = RC.STEAL_FOUL_BONUS * view.bonus5 + RC.TURNOVER_POINTS * view.tos
        return shot_value - banked >= self._bar(view, self._contested(view, play))
… lines 1322–1322: printed in the shared core (Part 1) …
    @staticmethod
    def _plan_kind(view, targets):
        """The plan, named once from the primary target (it changes only when the plan does): the carrier
        is the target (isolation), a post-up target (post feed), a rim target off the ball (cut to the
        rim), or a jump-shooter off the ball (swing to the shooter)."""
        if not targets:
            return None
        t = targets[0]
        if t['o'] == view.ball:
            return 'isolation'
        return {'post': 'post feed', 'layup': 'cut to the rim', 'dunk': 'cut to the rim',
                'floater': 'cut to the rim'}.get(t['shot'], 'swing to the shooter')
```

playmaker_engine/advantage.py

The six advantage states (step 3g).

```
"""Advantage states (step 3g): what an offence has created on the board, and for whom.

advantages(state, prev=None) is a pure function of what both players can see. Each advantage names the
attacker it benefits and is valued at the shot value after the defence's reaction that it opens (from
where the attacker stands, the same contest model as everywhere else). These are labels for the
computer's search and the post-game review, not scoring rules.

The six states:
  beaten       a defender is locked (lost a Dribble, or held by a Screen after a Challenge), and the
               attacker he was guarding has a shot he can't contest
  mismatch     an attacker's ▲ shot type meets a ▼ contest from the only defender who can reach it
  help_pulled  a defender moved toward the ball and away from the attacker he was nearest to, and that
               attacker now has no clean reachable contest (needs prev: the board before the move)
  bad_closeout the only defender who can reach the shot arrives with a foul roll, or with the -3 for
               being further from the rim than the shooter
  spent        the defence has no tokens, or the likely closeout defender (the nearest) has used his Move
  numbers      more attackers with a ▲ shot in their zone than defenders who can reach a clean contest
               on them in one reaction (the advantage goes to the best attacker left over)
"""
from . import board as B
from . import rules_config as RC

KINDS = ('beaten', 'mismatch', 'help_pulled', 'bad_closeout', 'spent', 'numbers')
WORDS = {'beaten': 'beat his man', 'mismatch': 'a mismatch', 'help_pulled': 'help pulled off him',
         'bad_closeout': 'a bad closeout', 'spent': 'a spent defence', 'numbers': 'numbers'}


def reaction_budget(state):
    return min(2, state.tok_x) if state.tok_x > 0 else 0


def reach_maps(state, reach=None):
    """Each defender's reach in the reaction (free and fixed-arrow squares), as the solver models it."""
    budget = reaction_budget(state)
    if reach is not None:
        return reach(state, budget)
    from .solver import defender_reach
    return [defender_reach(state, d, budget) for d in range(5)]

```

```
40
41  def contests(state, i, t, maps):
42      """Every defender who can contest attacker i shooting t from his square after the reaction, each at
43      his best: clean (facing the shooter), far (the -3), contest rating and the shot's value after it."""
44      from .scoring import foul_on_shot_points
45      spos = state.o[i]
46      pts = RC.ZONE_POINTS[B.zone(spos)]
47      card = state.ctx.off[i]
48      rules = state.ctx.rules
49      if rules.contest_geometry != 'current' or rules.arrow_mode != 'single':
50          return _contests_geo(state, i, t, maps, spos, pts, card)
51      out = []
52      for d in range(5):
53          free, fixed = maps[d]
54          best = None
55          for q in set(free) | set(fixed):
56              if not B.adjacent(q, spos) or q in state.o:
57                  continue
58              on = B.dir_between(q, spos)
59              far = B.dist_rim(q) > B.dist_rim(spos)
60              c = state.ctx.dfn[d].con(t) - (RC.FAR_CONTEST_PENALTY if far else 0)
61              if state.ctx.rules.floor_contest_zero:
62                  c = max(0, c)
63              v = max(0, pts * (card.shot(t) - c))
64              clean = q in free or on in fixed.get(q, {})
65              if not clean:
66                  pf = state.ctx.dfn[d].r['foul_contest'] / 20.0
67                  fp = float(foul_on_shot_points(card, t, pts, state.ctx.rules)[0])
68                  v = pf * fp + (1 - pf) * v
69              cand = dict(defender=d, clean=clean, far=far, contest=c, value=v, square=B.sq(q))
70              if best is None or (v, not clean) < (best['value'], not best['clean']):
71                  best = cand
72          if best:
73              out.append(best)
74      return sorted(out, key=lambda e: e['value'])
75
76
77  def _contests_geo(state, i, t, maps, spos, pts, card):
78      """contests() under a step 3h setting, through geometry.best_terms (far = Behind's -3)."""
79      from . import geometry as G
80      from .scoring import foul_on_shot_points
81      rules = state.ctx.rules
82      out = []
83      for d in range(5):
84          free, fixed = maps[d]
85          best = None
86          for q in set(free) | set(fixed):
87              if not B.adjacent(q, spos) or q in state.o:
88                  continue
89              opts = ([None] if q in free else []) + [(a,) for a in fixed.get(q, {})]
90              for arrows in opts:
91                  bt = G.best_terms(rules, q, arrows, spos)
92                  if bt is None:
93                      continue
94                  _, foul_roll, penalty, cat, _ = bt
95                  c = state.ctx.dfn[d].con(t) - (RC.FAR_CONTEST_PENALTY if penalty else 0)
96                  if rules.floor_contest_zero:
97                      c = max(0, c)
98                  v = max(0, pts * (card.shot(t) - c))
99                  if foul_roll:
100                     pf = state.ctx.dfn[d].r['foul_contest'] / 20.0
101                     fp = float(foul_on_shot_points(card, t, pts, rules)[0])
102                     v = pf * fp + (1 - pf) * v
103                 cand = dict(defender=d, clean=not foul_roll, far=penalty, contest=c, value=v, square=B.sq(q),
104                             category=cat)
105                 if best is None or (v, foul_roll) < (best['value'], not best['clean']):
106                     best = cand
107         if best:
108             out.append(best)
109     return sorted(out, key=lambda e: e['value'])
110
111
112 def looks(state, i, maps):
113     """Attacker i's shot types from his square: [(value after the reaction, type, contests)], best first."""
114     if not B.on_board(state.o[i]):
115         return []
116     z = B.zone(state.o[i])
117     if not RC.ZONE_SHOTS.get(z):
118         return []                               # plain wood: no shot from here
119     pts = RC.ZONE_POINTS[z]
120     out = []
121     for t in RC.ZONE_SHOTS[z]:
122         if state.ctx.three_only and t != 'three':
123             continue
124         cs = contests(state, i, t, maps)
125         v = cs[0]['value'] if cs else float(pts * state.ctx.off[i].shot(t))
126         out.append((v, t, cs))
127     return sorted(out, key=lambda x: -x[0])
128
129
130 def _adv(kind, attacker, value, shot, defender=None, **kw):
131     return dict(kind=kind, attacker=attacker, value=float(value), shot=shot, defender=defender, **kw)
132
133
134 def advantages(state, prev=None, maps=None):
135     """Every advantage on the board: a list of dicts (kind, attacker, value, shot, defender), best first.
136     prev is the board before the defence's last turn (for help_pulled); without it that state is skipped."""
```

```
    if state.stage == 'over' or not B.on_board(state.o[state.ball]):
        return []
    maps = maps if maps is not None else reach_maps(state)
    L = {i: looks(state, i, maps) for i in range(5)}
    off, dfn = state.ctx.off, state.ctx.dfn
    out = []
    nearest_x = lambda i: min(range(5), key=lambda j: (B.cheb(state.x[j], state.o[i]), j))
    # beaten: a locked defender, and the attacker he was guarding has a shot he can't contest
    for j in range(5):
        if not state.locked('X', j):
            continue
        beat = [o for o, x in state.beaten if x == j]
        a = beat[0] if beat else min(range(5), key=lambda i: (B.cheb(state.x[j], state.o[i]), i))
        if L[a]:
            v, t, cs = L[a][0]
            if not any(c['defender'] == j for c in cs):
                out.append(_adv('beaten', a, v, t, j))
    for i in range(5):
        if not L[i]:
            continue
        v, t, cs = L[i][0]
        # mismatch: a ▲ shot type against a ▼ contest from the only defender who can reach it
        for vt, tt, ct in L[i]:
            if off[i].bands['shot_' + tt] == 'H' and len(ct) == 1 and dfn[ct[0]['defender']].bands['con_' + tt] == 'L':
                out.append(_adv('mismatch', i, vt, tt, ct[0]['defender']))
                break
        # bad closeout: the only defender who can reach arrives with a foul roll or the -3
        if state.ctx.rules.contest_geometry != 'current':
            if cs and cs[0].get('category') in ('side', 'behind'):     # step 3h: the best contest isn't Front
                out.append(_adv('bad_closeout', i, v, t, cs[0]['defender'], foul_roll=not cs[0]['clean'],
                                far=cs[0]['far'], category=cs[0]['category']))
        elif len(cs) == 1 and (not cs[0]['clean'] or cs[0]['far']):
            out.append(_adv('bad_closeout', i, v, t, cs[0]['defender'], foul_roll=not cs[0]['clean'], far=cs[0]['far']))
        # spent: no tokens, or the likely closeout defender (the nearest) has used his Move
        j = nearest_x(i)
        if state.tok_x <= 0 or state.moved('X', j):
            out.append(_adv('spent', i, v, t, j, no_tokens=state.tok_x <= 0))
    # help pulled: a defender moved toward the ball and away from the attacker he was nearest to
    if prev is not None and prev.o == state.o:
        ball = state.o[state.ball]
        for j in range(5):
            if prev.x[j] == state.x[j]:
                continue
            man = min(range(5), key=lambda i: (B.cheb(prev.x[j], prev.o[i]), i))
            if man == state.ball or not L[man]:
                continue
            toward = B.cheb(state.x[j], ball) < B.cheb(prev.x[j], ball)
            away = B.cheb(state.x[j], state.o[man]) > B.cheb(prev.x[j], prev.o[man])
            v, t, cs = L[man][0]
            if toward and away and not any(c['clean'] for c in cs):
                out.append(_adv('help_pulled', man, v, t, j, helper=j))
    # numbers: more ▲ shooters in their zones than defenders who can reach a clean contest on them
    tri = {}
    for i in range(5):
        hs = [(vt, tt, ct) for vt, tt, ct in L[i] if off[i].bands['shot_' + tt] == 'H']
        if hs:
            tri[i] = (hs[0], {c['defender'] for vt, tt, ct in hs for c in ct if c['clean']})
    if tri:
        att = sorted(tri)
        cover = _max_cover(att, {i: tri[i][1] for i in att})
        if cover < len(att):
            # the attacker left over: the best one no full cover can include
            left = [i for i in att if _max_cover([a for a in att if a != i], {a: tri[a][1] for a in att}) == cover]
            if left:
                i = max(left, key=lambda a: tri[a][0][0])
                out.append(_adv('numbers', i, tri[i][0][0], tri[i][0][1], None, shooters=len(att), cover=cover))
    return sorted(out, key=lambda a: (-a['value'], KINDS.index(a['kind']), a['attacker']))


def _max_cover(att, can):
    """Most attackers that distinct defenders can each cleanly contest (bipartite matching)."""
    match = {}

    def aug(i, seen):
        for d in can.get(i, ()):
            if d in seen:
                continue
            seen.add(d)
            if d not in match or aug(match[d], seen):
                match[d] = i
                return True
        return False
    return sum(aug(i, set()) for i in att)


def describe(adv):
    """Plain words for the review: 'O1 has a mismatch on X4, for a three'."""
    who = f"O{adv['attacker'] + 1}"
    d = f" on X{adv['defender'] + 1}" if adv.get('defender') is not None and adv['kind'] != 'help_pulled' else ''
    if adv['kind'] == 'help_pulled':
        return f"Help came from X{adv['helper'] + 1}; {who} was left for a {adv['shot']}"
    return f"{who} has {WORDS[adv['kind']]}{d}, for a {adv['shot']}"
```

playmaker_engine/solver.py

The offence play generator (fallback candidates).

```
    def offence_plays(self, s):
        acts = legal_actions(s)
        out = []
        c = s.ball
        two = s.tok_o >= 2
        ev = self.ev
        budget_x = min(2, s.tok_x)
        maps = ev.reach(s, budget_x)
        for a in acts:
            if a.kind == 'shot':
                out.append(Play('shoot', (a,)))
            elif a.kind == 'pass':
                r = a.target
                look = ev.carrier_look(s, r, budget_x, True)[0] if B.on_board(s.o[r]) else -99
                out.append(Play('kick' if look >= 20 else 'swing', (a,)))
            elif a.kind == 'hesitate':
                out.append(Play('hesitate', (a,)))
            elif a.kind == 'dribble':
                out.append(Play('dribble', (a,)))
            elif a.kind == 'screen':
                tq = B.add(s.o[a.piece], B.DIRS[a.target])
                if tq in s.x:
                    out.append(Play('screen', (a,)))
        # drives: carrier movement, ranked by the look from the new square
        drives = []
        for a in acts:
            if a.kind in MOVEMENT and a.piece == c and B.on_board(a.dest):
                sh = s.with_(o=s.o[:c] + (a.dest,) + s.o[c + 1:])
                best = None
                for t in RC.ZONE_SHOTS[B.zone(a.dest)]:
                    v = ev.defended(sh, maps, c, a.dest, t, budget_x)[0]
                    if best is None or v > best[0]:
                        best = (v, t)
                drives.append((best[0] if best else -50, a, best[1] if best else None))
        drives.sort(key=lambda x: -x[0])
        for v, a, t in drives[:self.cfg.drive_k]:
            out.append(Play('drive', (a,)))
            if two and t is not None and not (s.ctx.hold_for_one and s.clock > RC.PHASE_SECONDS):
                out.append(Play('drive+shoot', (a, Action('shot', 'O', c, shot=t))))
        # relocations: off-ball movement to the best open shot squares
        rel = []
        for a in acts:
            if a.kind in MOVEMENT and a.piece != c and B.on_board(a.dest) and B.zone(a.dest) != '.':
                sh = s.with_(o=s.o[:a.piece] + (a.dest,) + s.o[a.piece + 1:])
                best = max((ev.defended(sh, maps, a.piece, a.dest, t, budget_x)[0] for t in RC.ZONE_SHOTS[B.zone(a.dest)]),
                           default=-50)
                rel.append((best, a))
        rel.sort(key=lambda x: -x[0])
        can_pass = s.count('O', 'pass') < RC.LIMITS['O']['pass']
        for v, a in rel[:self.cfg.relocate_k]:
            out.append(Play('relocate', (a,)))
            if two and can_pass:
                out.append(Play('relocate+kick', (a, Action('pass', 'O', c, target=a.piece))))
        # seal: move round your defender, then Screen with the T on it
        if two and s.count('O', 'screen') < RC.LIMITS['O']['screen']:
            for p in range(5):
                if p == c or s.is_screening(p) or not B.on_board(s.o[p]):
                    continue
                for j, xp in enumerate(s.x):
                    if not B.adjacent(xp, s.o[p]):
                        continue
                    for a in acts:
                        if a.kind == 'move' and a.piece == p and B.adjacent(a.dest, xp) and B.dist_rim(a.dest) < B.dist_rim(xp):
                            d = B.dir_between(a.dest, xp)
                            out.append(Play('seal', (a, Action('screen', 'O', p, target=d))))
                            break
        return out
```

computer_levels.json (offence)

The offence levels, habits and levers.

```
{
  "levels": {
    "rookie": {
      "concepts": {
        "reaction_shot": true,
        "plan": true,
        "patience": true,
        "drain": false,
        "risk": true,
        "lookahead": true
      },
      "habits": 1,
      "tunnel": 5,
      "reading": "exact",
      "lookahead": 1,
      "randomness": 0.03,
      "patience": {
        "open": {
          "12": 24,
          "10": 22,
          "8": 20,
          "6": 17,
          "4": 15,
          "2": 0
        },
```

```
        "contested": {
          "12": 33,
          "10": 30,
          "8": 26,
          "6": 21,
          "4": 17,
          "2": 0
        }
      },
      "shoot_by": 4,
      "pass_risk_tolerance": 0.5,
      "pass_alternatives": [
        "receiver"
      ],
      "plays": [
        "shoot",
        "dribble",
        "pass"
      ],
      "advantage_depth": 1
    },
    "starter": {
      "concepts": {
        "reaction_shot": true,
        "plan": true,
        "patience": true,
        "drain": true,
        "risk": true,
        "lookahead": true
      },
      "habits": 1,
      "tunnel": 5,
      "reading": "exact",
      "lookahead": 1,
      "randomness": 0.0,
      "patience": {
        "open": {
          "12": 27,
          "10": 25,
          "8": 22,
          "6": 19,
          "4": 16,
          "2": 0
        },
        "contested": {
          "12": 36,
          "10": 32,
          "8": 27,
          "6": 22,
          "4": 17,
          "2": 0
        }
      },
      "shoot_by": 0,
      "pass_risk_tolerance": 0.8,
      "pass_alternatives": [
        "receiver",
        "passer",
        "screen"
      ],
      "habits_not": [
        "forces_passes"
      ],
      "plays": [
        "shoot",
        "dribble",
        "pass",
        "screen",
        "speed",
        "seal"
      ],
      "advantage_depth": 2
    },
    "allstar": {
      "concepts": {
        "reaction_shot": true,
        "plan": true,
        "patience": true,
        "drain": true,
        "risk": true,
        "lookahead": true
      },
      "habits": 0,
      "tunnel": 5,
      "reading": "exact",
      "lookahead": 1,
      "randomness": 0.0,
      "patience": {
        "open": {
          "12": 27,
          "10": 25,
          "8": 22,
          "6": 19,
          "4": 16,
          "2": 0
        },
        "contested": {
```

```
          "12": 36,
          "10": 32,
          "8": 27,
          "6": 22,
          "4": 17,
          "2": 0
        }
      },
      "shoot_by": 0,
      "pass_risk_tolerance": 1.0,
      "pass_alternatives": [
        "receiver",
        "passer",
        "screen",
        "two_pass"
      ],
      "plays": [
        "shoot",
        "dribble",
        "pass",
        "screen",
        "speed",
        "seal",
        "cut",
        "move_advantage"
      ],
      "advantage_depth": 2
    }
  },
  "habits": [
    "settles_early",
    "never_screens",
    "ignores_mismatch",
    "star_only",
    "forces_passes"
  ],
  "plan_weight": 1.5,
  "_patience_about": "Shot bars by the clock (seconds left; a phase is two). A shot goes up only when its value, after the reaction, clears the bar and is worth at least what continuing is estimated to be worth. 'contested': a defender can still contest it; 'open': nobody can. The bars drop each phase; at the last chance there is no bar.",
  "loop_penalty": 2.0,
  "_loop_about": "loop_penalty: points off a non-shot play that ends where the offence already stood earlier this possession (its five squares and the ball), per earlier visit, up to three. shoot_by: from this clock on (Rookie: the phase before the last), the offence takes its best shot rather than keep building; 0 = only the last phase, which has no bar.",
  "_pass_about": "Step 3f. pass_risk_tolerance multiplies the -5 of a pick-off when a pass is valued (1.0 honest, below 1.0 reckless). pass_alternatives: what the level weighs against a contested pass, each valued with the next turn able to pass: receiver (a cut or relocation to open a lane), passer (move or dribble to change the angle), screen (a screen or seal), two_pass (two shorter clean passes). habits_not: forces_passes is a Rookie habit.",
  "_plays_about": "Step 3g. plays: the advantage plays the level knows (shoot, dribble attack, pass to move or finish an advantage, screen for a mismatch, speed burst, post seal, cut, and moving the advantage to the attacker the help left). advantage_depth: 1 = create only; 2 = create, the defence's best reply, then exploit.",
  "habit_words": {
    "settles_early": "lowered its shot bar a phase early, taking looks it could have waited on",
    "settles_for_first_shot": "took the first look above a low bar",
    "never_screens": "never set a screen",
    "ignores_mismatch": "passed by position order, not by matchup",
    "star_only": "always worked toward its highest-rated scorer",
    "forces_passes": "passed to its primary target whenever it could, however well the lane was guarded"
  },
  "concept_words": {
    "reaction": "didn't count the reaction to a declared shot",
    "reserve": "didn't keep a reaction in hand",
    "lookahead": "looked only at the shot on the board now",
    "help": "didn't send help off the ball",
    "walls": "didn't wall drives with the on-ball arrow",
    "lanes": "didn't point arrows across passing lanes",
    "plan": "didn't build toward a target",
    "patience": "didn't weigh waiting for a better shot",
    "drain": "didn't work to drain the defence's reactions",
    "risk": "ignored turnover risk"
  }
}
```

Appendix

playmaker_engine/policies.py

The base player class, and the greedy baseline computer (the "baseline" option, not offered on the setup screen).

```python
"""Agents and the game loop. No rule logic lives here: policies only pick from legal_actions."""
import random
from fractions import Fraction

from .actions import legal_actions, MOVEMENT
from .resolve import apply, sample, ChanceNode
from .scoring import score, is_terminal, shot_ev


class Policy:
    """choose(state) -> one of legal_actions(state)."""
    name = 'policy'

    def reset(self, seed):
        pass

    def choose(self, state):
```

```
        raise NotImplementedError


class RandomPolicy(Policy):
    name = 'random'

    def __init__(self, seed=0):
        self.rng = random.Random(seed)

    def reset(self, seed):
        self.rng = random.Random(seed)

    def choose(self, state):
        return self.rng.choice(legal_actions(state))


def expected(node, fn):
    """Expected value of fn over a node's outcomes."""
    if isinstance(node, ChanceNode):
        return sum(Fraction(p) * expected(s, fn) for p, _, s in node.outcomes)
    return Fraction(fn(node))


def immediate_shot_value(state):
    """Best value the ball carrier could get by shooting right now (no defensive reaction)."""
    from . import board as B
    from . import rules_config as RC
    if state.stage == 'over':
        return score(state).total
    p = state.o[state.ball]
    if not B.on_board(p):
        return 0
    types = RC.ZONE_SHOTS[B.zone(p)]
    if not types:
        return 0
    return max(shot_ev(state, state.ball, t) for t in types)


class GreedyPolicy(Policy):
    """Baseline. Offence: take the best shot available this turn (shoot now, or one
    movement action then shoot); otherwise drift toward a better shot. Defence: answer
    a pass with the best intercept, then pick the action that leaves the carrier the
    worst immediate shot (and contest a declared shot as well as it can)."""
    name = 'greedy'

    def choose(self, state):
        acts = legal_actions(state)
        if len(acts) == 1:
            return acts[0]
        if state.stage == 'target':
            return max(acts, key=lambda a: max(state.ctx.off[a.target].r[k] for k in state.ctx.off[a.target].r if k.startswith('shot
_')))
        if state.stage == 'forced':
            return max(acts, key=lambda a: expected(apply(state, a), lambda n: min(
                (expected(apply(n, c), immediate_shot_value) for c in legal_actions(n)), default=Fraction(0))
                if n.stage == 'contest' else immediate_shot_value(n)))
        if state.side == 'O':
            return self._offence(state, acts)
        return self._defence(state, acts)

    def _offence(self, state, acts):
        shots = [a for a in acts if a.kind == 'shot']
        best_now = max(shots, key=lambda a: shot_ev(state, a.piece, a.shot)) if shots else None
        v_now = shot_ev(state, best_now.piece, best_now.shot) if best_now else None
        best_mv, v_mv = None, None
        for a in acts:
            if a.kind not in MOVEMENT or a.piece != state.ball:
                continue
            nxt = apply(state, a)
            if isinstance(nxt, ChanceNode) or nxt.side != 'O' or nxt.stage != 'play':
                continue
            v = immediate_shot_value(nxt)
            if v_mv is None or v > v_mv:
                best_mv, v_mv = a, v
        if best_now and (v_mv is None or v_now >= v_mv):
            return best_now
        if best_mv is not None:
            return best_mv
        if best_now:
            return best_now
        for kind in ('end_turn', 'hesitate', 'pass'):
            for a in acts:
                if a.kind == kind:
                    return a
        return acts[0]

    def _defence(self, state, acts):
        icpt = [a for a in acts if a.kind == 'intercept']
        if icpt:
            o = state.ctx.off[state.pending[0]]
            return min(icpt, key=lambda a: o.r['pass'] - state.ctx.dfn[a.piece].r['intercept'])
        best, best_v = None, None
        for a in acts:
            nxt = apply(state, a)
            v = expected(nxt, immediate_shot_value)
            if a.kind not in ('end_turn',):
                v += Fraction(1, 1000)        # spend tokens only for a real gain
```

```
            if best_v is None or v < best_v:
                best, best_v = a, v
        return best


POLICIES = {'random': RandomPolicy, 'greedy': GreedyPolicy}


def play(setup, offence_policy, defence_policy, seed=0, max_steps=5000, check=None):
    """Play one possession. setup: a GameState or a scenario number.
    Returns dict(log=[...], score=Score, final=GameState, states=n). Same seed, same game.
    check: optional callable(state) run after every apply (invariants)."""
    from .scenarios import scenario_state
    state = scenario_state(setup) if isinstance(setup, int) else setup
    rng = random.Random(seed)
    offence_policy.reset(seed * 2 + 1)
    defence_policy.reset(seed * 2 + 2)
    log = []
    steps = 0
    while not is_terminal(state):
        steps += 1
        if steps > max_steps:
            raise RuntimeError('game did not terminate')
        pol = offence_policy if state.side == 'O' or state.stage == 'target' else defence_policy
        a = pol.choose(state)
        before = state
        nxt = apply(state, a)
        label = ''
        if isinstance(nxt, ChanceNode):
            nxt, label = sample(nxt, rng)
        log.append((before.clock, before.phase, str(a), label))
        state = nxt
        if check:
            check(before, a, state)
    return dict(log=log, score=score(state), final=state, steps=steps)


def format_log(result):
    lines = []
    for clock, phase, a, label in result['log']:
        lines.append(f'[phase {phase}, clock {clock:>2}] {a}' + (f'  -> {label}' if label else ''))
    lines.append(f'Score: {result["score"]}')
    return '\n'.join(lines)
```

playmaker_engine/solver.py

The solver's search and the exact endgame solver. A computer level uses it only with lookahead "solver", which no level uses at present.

```
class NodeCap(Exception):
    pass
```

… lines 991–992: printed in the shared core (Part 1) …

```
def _terminal_turn(s):
    """Is this the defence's last turn before the possession resolves (contest or forced shot)?"""
    if not (s.side == 'X' and not s.pending and s.inbound == 0 and s.ctx.rules.contest_choice in ('min_value', 'player')):
        return False
    if s.stage == 'reaction':
        return True
    if s.stage != 'play' or s.clock > RC.PHASE_SECONDS:
        return False
    # the final turn, or the defence's last token (which ends the phase at once, then the clock runs out)
    return s.final or (s.tok_x == 1 and s.ctx.rules.defence_empty_ends_phase)
```

… lines 1003–1007: printed in the shared core (Part 1) …

```
def pruned_actions(s):
    """Legal actions, with provably irrelevant defence actions removed on its last turn.
    On that turn only the defenders next to the shooter, their arrows and Steals matter:
      * second actions that don't end a defender next to the shooter (or turn an adjacent
        defender's arrow, or Steal) can only remove contest options, never add one;
      * first actions by defenders that start and end far from the shooter can't affect it
        (they can't reach it, and can't unblock anyone who can); one is kept as a no-op."""
    acts = legal_actions(s)
    if not _terminal_turn(s):
        return acts
    sp = s.o[s.ball]
    keep, noop = [], None
    for a in acts:
        if a.kind in ('steal', 'end_turn', 'forfeit'):
            keep.append(a)
        elif a.kind in MOVEMENT:
            start = s.x[a.piece]
            if s.turn_n >= 1 or s.tok_x <= 1:          # this is the defence's last action
                if B.adjacent(a.dest, sp):
                    keep.append(a)
                elif noop is None and s.turn_n == 0 and not B.adjacent(start, sp):
                    noop = a                           # a true no-op: nobody near the shooter moves
            elif B.cheb(a.dest, sp) <= REACH_BOUND or B.cheb(start, sp) <= REACH_BOUND:
                keep.append(a)
            elif noop is None:
                noop = a
        elif s.turn_n == 0 and noop is None:
            noop = a
    if noop is not None and not any(a.kind == 'end_turn' for a in keep):
        keep.append(noop)
    return keep
```

… lines 1039–1040: printed in the shared core (Part 1) …

```
def exact_solve(s, cap=10 ** 6, prune=True, tt=None, stats=None):
    """Exact best play from s (target as set in s): expectimax over raw engine actions with a
```

```
    transposition table. Returns (Val, {action: Val}) for the root. Raises NodeCap past `cap` applies."""
    tt = {} if tt is None else tt
    count = [0]

    def node_value(node):
        if isinstance(node, ChanceNode):
            return mix([(float(p), (node_value(sub),)) for p, _, sub in node.outcomes])[0]
        return rec(node)

    def rec(st):
        if st.stage == 'over':
            return terminal_vals(st, (None,))[0]
        v = tt.get(st)
        if v is not None:
            return v
        acts = pruned_actions(st) if prune else legal_actions(st)
        vals = []
        for a in acts:
            count[0] += 1
            if count[0] > cap:
                raise NodeCap()
            vals.append(node_value(apply(st, a)))
        maximise = st.side == 'O' or st.stage in ('target', 'pick')
        v = max(vals, key=lambda x: x.ev) if maximise else min(vals, key=lambda x: x.ev)
        tt[st] = v
        return v

    if s.stage == 'over':
        return terminal_vals(s, (None,))[0], {}
    acts = pruned_actions(s) if prune else legal_actions(s)
    per = {}
    for a in acts:
        count[0] += 1
        if count[0] > cap:
            raise NodeCap()
        per[a] = node_value(apply(s, a))
    maximise = s.side == 'O' or s.stage in ('target', 'pick')
    v = max(per.values(), key=lambda x: x.ev) if maximise else min(per.values(), key=lambda x: x.ev)
    if stats is not None:
        stats['applies'] = count[0]
    return v, per
```

… lines 1084–1089: printed in the shared core (Part 1) …

```
@dataclass
class Entry:
    vals: tuple
    best: Play = None
    children: list = field(default_factory=list)     # [(Play, vals)]
```

… lines 1095–1096: printed in the shared core (Part 1) …

```
class Searcher:
    def __init__(self, cfg=DEFAULT, targets=(None,), seed=0, deadline=None):
        self.cfg = cfg
        self.targets = targets
        self.ev = Evaluator(cfg, targets)
        self.gen = Generator(cfg, self.ev)
        self.rng = random.Random(seed) if seed else None
        self.deadline = deadline
        self.tt = {}
        self.nodes = 0
        self.exact_hits = 0
        self.exact_tries = 0
        self.exact_tt = {}
        self.root_depth = 0
        self.concept_counts = {}

    def choose(self, children, maximise):
        """Offence: best per target (it knows its own target). Defence: the play that
        minimises the offence's best value across targets."""
        if maximise:
            if len(self.targets) == 1:
                p, v = max(children, key=lambda pv: pv[1][0].ev)
                return v, p
            out = []
            for t in range(len(self.targets)):
                out.append(max(children, key=lambda pv: pv[1][t].ev)[1][t])
            p = max(children, key=lambda pv: max(x.ev for x in pv[1]))[0]
            return tuple(out), p
        p, v = min(children, key=lambda pv: max(x.ev for x in pv[1]))
        return v, p

    def in_exact_range(self, s):
        n = self.cfg.endgame_tokens
        return (len(self.targets) == 1 and s.stage == 'play' and s.inbound == 0 and s.clock <= RC.PHASE_SECONDS
                and s.tok_o <= n and s.tok_x <= n and not s.ctx.coach_call)

    def value(self, s, depth):
        if s.stage == 'over':
            return terminal_vals(s, self.targets)
        if self.deadline and time.time() > self.deadline:
            raise Timeout()
        k = (s, depth)
        e = self.tt.get(k)
        if e is not None:
            return e.vals
        self.nodes += 1
        if self.in_exact_range(s):
            self.exact_tries += 1
            try:
```

```
                v, _ = exact_solve(s, cap=self.cfg.exact_node_cap, tt=self.exact_tt)
                self.exact_hits += 1
                vals = (v,)
                self.tt[k] = Entry(vals)
                return vals
            except NodeCap:
                pass
        if depth <= 0:
            if self.cfg.quiescence and s.side == 'X' and s.stage == 'play':
                return self.quiesce(s)
            return self.ev(s)
        shallow = (self.root_depth - depth) < self.cfg.raw_depth
        plays = self.gen.plays(s, shallow, self.rng)
        maximise = (s.side == 'O' and s.stage != 'reaction') or s.stage in ('target', 'pick', 'forced')
        children = []
        for p in plays:
            node = apply_play(s, p)
            if node is None:
                continue
            children.append((p, node))
        if not children:
            return self.ev(s)
        width = self.cfg.root_beam if depth == self.root_depth else self.cfg.beam
        if len(children) > width and depth >= 2:
            scored = [(expected(n, self.ev), p, n) for p, n in children]
            scored.sort(key=lambda x: (-x[0][0].ev if maximise else max(v.ev for v in x[0])))
            children = [(p, n) for _, p, n in scored[:width]]
        results = []
        for p, node in children:
            v = self.expect(node, depth - 1)
            results.append((p, v))
        vals, best = self.choose(results, maximise)
        self.tt[k] = Entry(vals, best, results)
        return vals

    def quiesce(self, s):
        """Leaf with the defence to move: let it take its turn (best of its concept plays), then evaluate."""
        k = (s, -1)
        e = self.tt.get(k)
        if e is not None:
            return e.vals
        plays = self.gen.plays(s, False, light=True)
        results = []
        for p in plays:
            node = apply_play(s, p)
            if node is None:
                continue
            results.append((p, expected(node, self.ev)))
        if not results:
            vals = self.ev(s)
        else:
            results.sort(key=lambda pv: max(v.ev for v in pv[1]))
            vals, best = self.choose(results[:self.cfg.quiescence_beam], False)
        self.tt[k] = Entry(vals)
        return vals

    def expect(self, node, depth):
        if isinstance(node, ChanceNode):
            return mix([(float(p), self.expect(sub, depth)) for p, _, sub in node.outcomes])
        return self.value(node, depth)

    def search(self, s, budget_s=None, max_depth=None, min_depth=1):
        """Iterative deepening. Returns dict with vals, depth reached, best play, timings."""
        max_depth = max_depth or self.cfg.max_depth
        t0 = time.time()
        self.deadline = (t0 + budget_s) if budget_s else None
        done = None
        for d in range(1, max_depth + 1):
            self.root_depth = d
            try:
                vals = self.value(s, d)
            except Timeout:
                break
            done = (d, vals, self.tt.get((s, d)))
            if self.deadline is None and d >= max_depth:
                break
            if s.stage == 'over':
                break
        if done is None:                        # nothing finished: fall back to depth 1 without a deadline
            self.deadline = None
            self.root_depth = min_depth
            vals = self.value(s, min_depth)
            done = (min_depth, vals, self.tt.get((s, min_depth)))
        d, vals, entry = done
        return dict(depth=d, vals=vals, entry=entry, seconds=time.time() - t0, nodes=self.nodes,
                    evals=self.ev.calls, exact_hits=self.exact_hits, exact_tries=self.exact_tries)

    # ---- reading the result ----
    def principal_line(self, s, depth, target_index=0, max_turns=20):
        """Best plays from s, following the most likely outcome of each roll."""
        lines = []
        node = s
        d = depth
        while d > 0 and len(lines) < max_turns:
            if isinstance(node, ChanceNode):
                break
            if node.stage == 'over':
```

```
                break
            e = self.tt.get((node, d))
            if e is None or e.best is None:
                break
            p = e.best
            if len(self.targets) > 1 and (node.side == 'O' and node.stage == 'play'):
                p = max(e.children, key=lambda pv: pv[1][target_index].ev)[0]
            side = node.side
            nxt = apply_play(node, p)
            rolls = []
            while isinstance(nxt, ChanceNode):
                po, lab, sub = max(nxt.outcomes, key=lambda o: o[0])
                rolls.append((nxt.what, lab, float(po)))
                nxt = sub
            lines.append(dict(side=side, concept=p.concept, text=p.text(), rolls=rolls, play=p,
                              clock=node.clock, phase=node.phase))
            node = nxt
            d -= 1
        return lines, node
```

… lines 1262–1263: printed in the shared core (Part 1) …

```
def describe_terminal(node):
    if not isinstance(node, ChanceNode) and node.stage == 'over':
        return str(score(node))
    return ''
```